



Universidade Federal do ABC
Centro de Engenharia, Modelagem e Ciências Sociais Aplicadas
Engenharia de Informação

Sistema de Detecção de Pedestres Baseada em Visão Computacional para Semáforos Inteligentes

Lucas Pereira de Medeiros

Santo André - SP, Julho de 2026

Lucas Pereira de Medeiros

Sistema de Detecção de Pedestres Baseada em Visão Computacional para Semáforos Inteligentes

Trabalho de Graduação apresentado ao curso de Engenharia de Informação, como parte dos requisitos necessários para a obtenção do Título de Bacharel em Engenharia de Informação

Universidade Federal do ABC – UFABC
Centro de Engenharia, Modelagem e Ciências Sociais Aplicadas
Engenharia de Informação

Orientador: Prof. Dr. Celso Setsuo Kurashima

Santo André - SP
Julho de 2026

Dedico este trabalho ao meu passado eu.

AGRADECIMENTOS

Gostaria de agradecer ao professor Dr. Celso Kurashima pelo acompanhamento e apoio ao projeto e à Universidade Federal do ABC pela oportunidade de ensino e aprendizado.

Agradeço também aos meus pais e avós que sempre me apoiaram durante minha jornada acadêmica que, sem eles, poderia não ter tomado o mesmo rumo.

*“Se eu vi mais longe,
foi por estar sobre
ombros de gigantes.”
(Isaac Newton)*

RESUMO

O presente trabalho apresenta o desenvolvimento e a validação de um protótipo de sistema semafórico inteligente baseado em visão computacional, motivado pelo cenário onde os semáforos ainda operam predominantemente com temporizações fixas e sem considerar o mundo a sua volta. A proposta consiste na integração de um sistema de aquisição de imagens, um módulo de processamento baseado em inteligência artificial e um controlador lógico programável capaz de ajustar dinamicamente o ciclo semafórico em ambiente de laboratório. A arquitetura desenvolvida é composta por três módulos principais, sendo eles o sistema visual, sistema de processamento e controlador semafórico. O sistema visual, baseado no microcontrolador *ESP32-CAM*, foi configurado para operar como câmera IP, transmitindo continuamente imagens via *Ethernet*. O módulo de processamento, implementado em linguagem *Python* utilizando a biblioteca do *OpenCV* e a rede neural *YOLOv8*, realiza a detecção de pedestres nas imagens recebidas e transmite o resultado ao controlador por meio do protocolo *Modbus/TCP*. O controlador, desenvolvido em linguagem *Ladder* sobre a plataforma *OpenPLC* em um microcontrolador *ESP32*, interpreta essas informações para comandar o ciclo semafórico, incorporando ainda mecanismos de supervisão e monitoramento do estado da câmera para adoção automática de um modo seguro em caso de falhas de comunicação. Os resultados obtidos demonstraram o funcionamento integrado dos três módulos, validando a transmissão de imagens, a detecção de pedestres, a comunicação em tempo real e a atuação automática do controlador. Embora o sistema ainda apresente limitações relacionadas ao desempenho dos componentes e à precisão do modelo de detecção, os experimentos comprovaram a viabilidade da proposta, evidenciando que a integração entre visão computacional, inteligência artificial e controladores programáveis constitui uma alternativa promissora para o desenvolvimento de sistemas inteligentes de controle de tráfego urbano.

Palavras-chave: Visão computacional; Eletrônica; Linguagem de programação; Inteligência artificial; Microcontroladores; Sistemas embarcados; Protocolos de rede.

ABSTRACT

This work presents the development and validation of a computer vision-based intelligent traffic light system prototype, motivated by the fact that traffic lights predominantly operate with fixed timings without considering their surroundings. The proposal involves integrating an image acquisition system, an AI-based processing module, and a programmable logic controller capable of dynamically adjusting the traffic light cycle in a laboratory setting. The developed architecture comprises three main modules: the visual system, the processing system, and the traffic light controller. The visual system, based on the ESP32-CAM microcontroller, was configured to operate as an IP camera, continuously transmitting images via Ethernet. The processing module—implemented in Python using the OpenCV library and the YOLOv8 neural network—detects pedestrians in the received images and transmits the results to the controller via the Modbus/TCP protocol. The controller, developed using Ladder Logic on the OpenPLC platform running on an ESP32 microcontroller, interprets this information to manage the traffic light cycle; it also incorporates supervision and camera-status monitoring mechanisms to automatically switch to a safe mode in the case of communication failures. The obtained results demonstrated the integrated operation of the three modules, validating image transmission, pedestrian detection, real-time communication, and automatic controller actuation. Although the system still exhibits limitations regarding component performance and detection model accuracy, the experiments confirmed the proposal's viability, highlighting that integrating computer vision, artificial intelligence, and programmable controllers offers a promising alternative for developing intelligent urban traffic control systems.

Keywords: Computer vision; Electronics; Programming languages; Artificial intelligence; Microcontrollers; Embedded systems; Network protocols.

SUMÁRIO

1	INTRODUÇÃO	1
1.1	Motivação	1
1.2	Objetivos	3
2	FUNDAMENTAÇÃO TEÓRICA	4
2.1	Sistemas semafóricos urbanos e controle de tráfego	4
2.2	Visão computacional e detecção de objetos	6
2.3	Controladores lógicos programáveis e OpenPLC	8
2.4	Comunicação industrial e protocolo Modbus/TCP	9
3	METODOLOGIA	11
3.1	Estudo de Caso	11
3.2	Projeção da Arquitetura Global do Sistema	13
3.3	Desenvolvimento do Sistema Visual	17
3.4	Desenvolvimento do Sistema de Processamento	20
3.5	Desenvolvimento do Controlador Semafórico	22
3.6	Integração entre os Módulos	28
3.7	Instalação e Execução do Programa de Processamento	29
4	RESULTADOS E DISCUSSÃO	33
4.1	Módulo visual	34
4.2	Módulo de processamento	34
4.3	Módulo de controle	37
4.4	Desafios	37
4.5	Custos de implementação	38
4.6	Considerações finais	40
5	CONCLUSÃO	42
	REFERÊNCIAS	43
	APÊNDICES	46
	APÊNDICE A – CÓDIGO DA ESP32-CAM	47
	APÊNDICE B – CÓDIGO DO MÓDULO DE PROCESSAMENTO	52

APÊNDICE C – LÓGICA LADDER 65

1 INTRODUÇÃO

O avanço das tecnologias de automação e inteligência artificial tem impulsionado o desenvolvimento significativo nas infraestruturas urbanas, principalmente na área da mobilidade e do controle de tráfego. Em centros urbanos cada vez mais densos, sistemas semafóricos baseados em temporizações estáticas mostram limitação estrutural para responder às variações dinâmicas do fluxo de veículos e de pedestres, gerando atrasos, aumento do consumo de combustível e impactos ambientais decorrentes das emissões de gás carbônico (World Bank, 2020; Organisation for Economic Co-operation and Development, 2019). Estudos em mobilidade urbana demonstram que a eficiência dos controladores de tráfego tem influência direta na fluidez viária, no tempo de deslocamento e na qualidade de vida urbana (VASCONCELLOS, 2014). Nesse cenário, soluções inteligentes passam a empregar sensores, sistemas de comunicação e algoritmos de decisão para oferecer maior adaptabilidade às condições reais do trânsito (HARRISON; ABBOTT, 2015).

Com base nesse contexto, o presente trabalho propõe a criação de um protótipo de sistema semafórico reativo, capaz de detectar a presença de pedestres através de câmeras e, por meio de visão computacional, ajustar automaticamente o tempo de sinalização de um controlador semafórico. Dessa forma, o estudo a seguir visa demonstrar a viabilidade de integração entre sistemas de inteligência artificial e controladores lógicos programáveis (CLPs) utilizados em sistemas semafóricos reais para a otimização de processos urbanos.

1.1 Motivação

O aumento da quantidade de veículos nas cidades brasileiras, associado à expansão urbana desordenada e à ineficiência de medidas de gestão inteligente do tráfego, traz à tona a necessidade de melhores soluções para o controle do tráfego. São Bernardo do Campo, município escolhido como referência para o cenário de estudo, apresenta perfil semelhante ao de outras cidades da Região Metropolitana de São Paulo, caracterizado por redes viárias congestionadas, longos tempos de espera e a dependência de intervenções manuais por agentes de trânsito em horários de pico, conforme possível notar na Figura 1. A utilização predominante de semáforos de tempo fixo, sem mecanismos de adaptação à demanda real, implica em grandes perdas operacionais, sobretudo quando o fluxo veicular e a demanda de travessia de pedestres variam ao longo do dia. (EMTU – Empresa Metropolitana de Transportes Urbanos, 2022)

Diante desse cenário, surge a oportunidade de aplicar conceitos de automação e visão computacional, uma vez que modelos modernos são capazes de detectar e classificar objetos em tempo real com alta precisão. Esses avanços abrem espaço para a criação de

sistemas de controle semafórico adaptativos utilizando conceitos oferecidos pelo curso de Engenharia de Informação da Universidade Federal do ABC.



Figura 1 – Presença de agentes de trânsito para coordenação dos semáforos. Fonte: Autoria própria.

Nesse contexto, destaca-se um trabalho recente que analisa a operação de um cruzamento na cidade de Limeira (SP), entre as avenidas Dr. Fabrício Vampré e Cônego Manoel Alves, que propõe um sistema inteligente de controle semafórico baseado na identificação e contagem de veículos por meio de técnicas de visão computacional e na posterior simulação de tempos de verde, amarelo e vermelho em ambiente de software e na plataforma Wokwi (simulador online de Arduino e outras plataformas) (XIMENES et al., 2024). Assim como no presente projeto, esse estudo utiliza recursos, como módulos *ESP32-CAM* e bibliotecas em *Python* (*OpenCV*), e busca ajustar a temporização semafórica a partir de informações extraídas de vídeo, explorando a viabilidade de soluções acessíveis para cidades de médio porte. Por outro lado, existem diferenças importantes quanto ao escopo e à abordagem, enquanto o projeto em questão concentra-se na contagem de veículos a partir de gravações e na variação do tempo de verde em função do fluxo, com tratamento apenas indireto dos pedestres, este projeto opta por um modelo de detecção binária de presença de pedestres e veículos em tempo real, a integração explícita da visão computacional a um controlador semafórico programável baseado em *OpenPLC* e

ESP32 e a representação, em escala de laboratório, de uma arquitetura mais próxima dos controladores utilizados em campo. Dessa forma, o presente estudo se conecta com a literatura existente, mas se compromete a demonstrar a viabilidade da integração entre a teoria e o que já existe.

1.2 Objetivos

O objetivo geral deste trabalho é projetar e implementar um protótipo funcional de sistema semafórico inteligente baseado em visão computacional, demonstrando sua capacidade de ser implementada em cenários urbanos reais. Busca-se comprovar que, mesmo utilizando hardwares como microcontroladores *ESP32*, câmeras compactas e softwares abertos, é possível estruturar uma solução inspirada em práticas modernas de controle semafórico inteligente, conforme discutido em literatura sobre cidades inteligentes (HARRISON; ABBOTT, 2015).

Entre os objetivos específicos, incluem-se a implementação de um algoritmo de detecção de pedestres, a integração entre esse sistema de detecção e um controlador lógico responsável por executar a lógica semafórica, a construção de um protótipo físico para fins de demonstração, a adoção de modos distintos de operação para o semáforo e a análise preliminar dos benefícios e limitações de uma eventual adoção desse tipo de solução em ambientes reais. Espera-se, ao final, que este projeto contribua para a discussão sobre a modernização do tráfego em cidades de médio porte e demonstre a viabilidade de estratégias de controle adaptativo baseadas em visão computacional.

2 FUNDAMENTAÇÃO TEÓRICA

O capítulo a seguir tem como objetivo contextualizar, do ponto de vista conceitual e tecnológico, os elementos que compõem o protótipo do sistema semafórico inteligente. A proposta se baseia na integração entre sistemas semafóricos urbanos, controladores lógicos programáveis, visão computacional, plataformas embarcadas e protocolos de comunicação industriais.

A seguir, serão discutidos as bases teóricas que sustentam o projeto, os conceitos de sinalização semafórica urbana e controle de tráfego, a arquitetura e os princípios de operação de controladores lógicos programáveis, os fundamentos de visão computacional e de modelos do tipo *YOLO* para detecção de objetos, os princípios de comunicação industrial baseados em protocolos *Modbus/TCP* e os aspectos relacionados às plataformas embarcadas *ESP32-CAM*.

2.1 Sistemas semafóricos urbanos e controle de tráfego

A sinalização semafórica é um sistema da sinalização das vias responsável por organizar os movimentos de veículos e pedestres por meio de indicações luminosas acionadas de forma alternada ou intermitente, de acordo com critérios de segurança operacional definidos em normas e manuais técnicos. No contexto brasileiro, o Manual Brasileiro de Sinalização de Trânsito estabelece conceitos, princípios de programação e critérios de implantação de semáforos, incluindo definições de tempos mínimos de verde, intervalos de amarelo, tempos de entreverdes, chamados *all-red*, e fases específicas para pedestres (Brasil. Departamento Nacional de Trânsito, 2014; CET – Companhia de Engenharia de Tráfego, 2026).

Historicamente, grande parte dos controladores instalados em cidades como São Paulo e São Bernardo do Campo opera em regime de tempo fixo, baseado em planos de tempo pré-determinados por horário e dia da semana, com pouca ou nenhuma influência direta de sensores adaptativos de tráfego (CET – Companhia de Engenharia de Tráfego, 2014). Nesses sistemas, os tempos de verde, amarelo e vermelho são calculados a partir de volumes médios de tráfego e parâmetros de projeto, sendo reproduzidos de forma repetitiva ao longo do dia. Essa abordagem simplifica a implementação, mas gera ineficiência em cenários de fluxo variável, como períodos de baixo movimento, quando veículos e pedestres permanecem aguardando em fases vermelhas mesmo na ausência de pedestres.

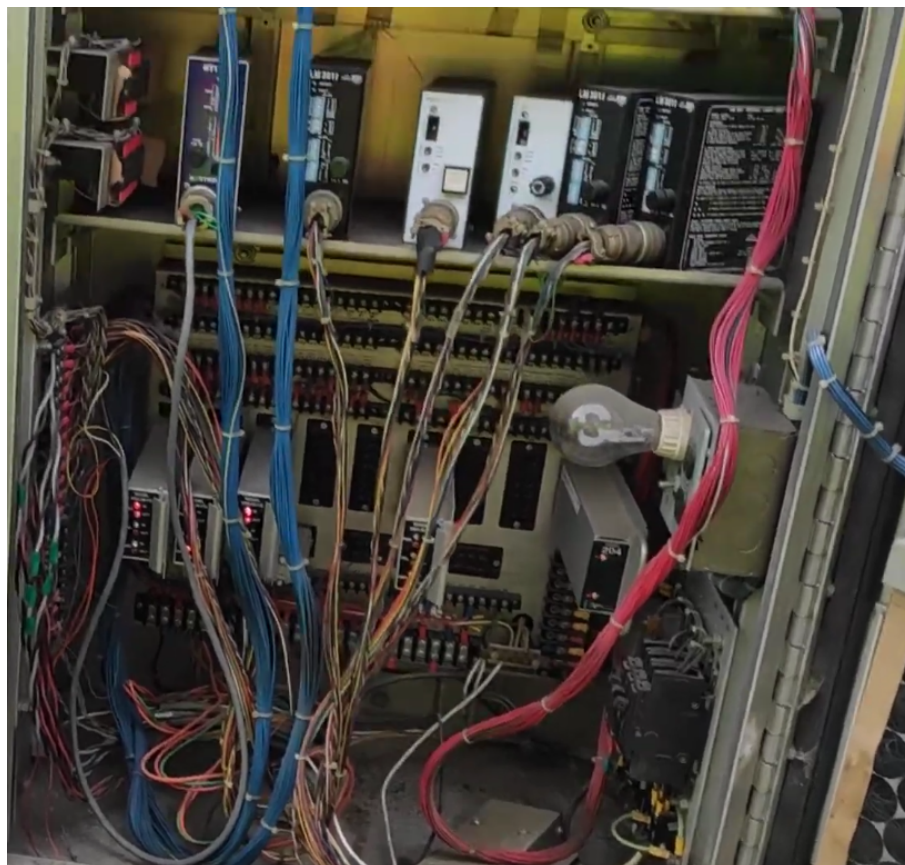


Figura 2 – Interior de um controlador semafórico. Fonte: ([QURISHEE, 2020](#))

Em contrapartida, padrões internacionais como a família *NTCIP 1202*, voltados a controladores de sinais atuados, introduzem o conceito de detecção e atuação dinâmica com base em sensores de presença e demanda, permitindo a modificação adaptativa dos tempos de verde de acordo com o estado da via ([NTCIP, 2019](#)). Nesses modelos, o controlador recebe sinais binários de sensores de laço indutivo, botoeiras de pedestres, detectores de vídeo ou outros dispositivos, e utiliza essas informações para estender, reduzir ou chamar certas fases, respeitando limites mínimos e máximos de tempo por segurança. A arquitetura proposta neste projeto se inspira nesse paradigma ao separar o nível de detecção, realizado por visão computacional, do nível de decisão, implementado em um computador e microcontrolador que simula um *CLP* e que interpreta apenas variáveis booleanas de presença de pedestres. Um exemplo de funcionamento de semafórico pode ser visualizado no material disponibilizado por Murad Al Qurishee ([QURISHEE, 2020](#)) e na Figura 2.

Ao reproduzir em escala de laboratório as noções de ciclo padrão, ciclo veicular estendido e travessia de pedestres condicionada à presença detectada, o protótipo busca aproximar a lógica de um controlador semafórico real de forma simplificada. Porém, ao mesmo tempo, mantendo a compatibilidade conceitual com recomendações técnicas de órgãos de trânsito, como a necessidade de tempos mínimos de verde, modos de alerta/se-

gurança e priorização da segurança pedestre-veículo ([Brasil. Departamento Nacional de Trânsito, 2014](#); [CET – Companhia de Engenharia de Tráfego, 2026](#)).

2.2 Visão computacional e detecção de objetos

Visão computacional é a área da inteligência artificial que trata da extração automática de informação semântica a partir de imagens e vídeos, englobando tarefas como detecção, classificação, segmentação e rastreamento de objetos ([GONZALEZ; WOODS, 2018](#)). Modelos modernos baseados em redes neurais convolucionais têm se mostrado altamente eficazes na detecção em tempo real de múltiplos objetos, incluindo veículos e pedestres, viabilizando aplicações em transporte inteligente, vigilância e automação de tráfego.

Entre as arquiteturas de detecção em tempo real, destacam-se os modelos da família *YOLO* (*You Only Look Once*), que formulam a detecção como um problema de regressão única, permitindo localizar e classificar objetos em uma única passagem da imagem pela rede. A versão *YOLOv8n*, desenvolvida pela Ultralytics, representa uma evolução em termos de eficiência e flexibilidade, oferecendo variantes de diferentes portes que equilibram precisão e custo computacional. Essa variante, em particular, é otimizada para execução em unidades centrais de processamento (*CPUs*) e dispositivos de menor capacidade, mantendo desempenho suficiente para detecção de pessoas em cenários urbanos simplificados ([Ultralytics, 2024](#)). A Figura 3 exemplifica o uso do *YOLO* para detecção de objetos em imagens.

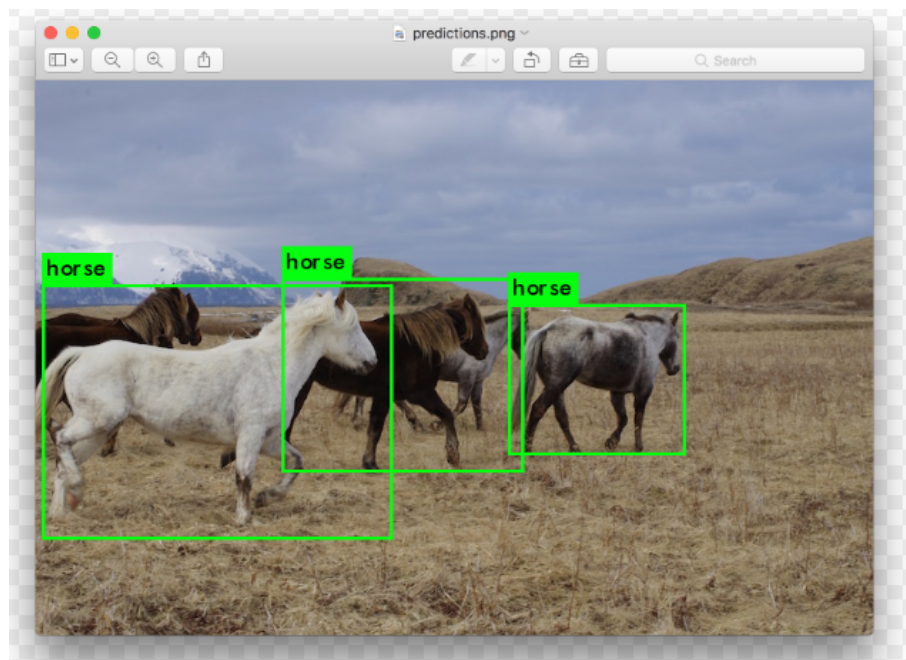


Figura 3 – Exemplo de utilização do YOLO. Fonte: ([REDMON, 2018](#))

A detecção de objetos em tempo real pode ser combinada a bibliotecas como *OpenCV*, que fornece ferramentas para captura de vídeo, pré-processamento de imagens e visualização de resultados. No contexto deste trabalho, o modelo *YOLOv8n* pré-treinado em *COCO* (conjunto de dados da Microsoft usado para treinar IAs na identificação e segmentação de objetos em imagens) é utilizado apenas em modo de inferência, sem necessidade de treinamento adicional e atuando de modo que, dada uma imagem, retorna uma lista de objetos detectados com classes e coordenadas ([Ultralytics, 2024](#)). Inicialmente, espera-se que ele proporcione um desempenho compatível com processamento próximo ao tempo real, em razão de sua arquitetura compacta e menor demanda computacional em comparação às variantes de maior porte da família YOLO. Caberá ao algoritmo interpretar essas detecções de forma a produzir duas variáveis booleanas simplificadas. Essa diminuição de perspectiva, de uma visão detalhada para variáveis binárias, é coerente com a arquitetura adotada, na qual o controlador de um sistema semafórico não precisa ter conhecimento direto de imagens, mas apenas de eventos de presença equivalentes aos fornecidos por sensores tradicionais. Também é recomendado pela literatura de visão computacional o uso de mecanismos de filtragem para evitar que ruídos, oclusões momentâneas ou detecções espúrias produzam respostas instáveis em sistemas de controle ([GONZALEZ; WOODS, 2018](#)). De modo semelhante, a aplicação de um pequeno tempo de *debounce* (intervalo de tempo usado para evitar excesso de requisições em softwares) para a presença de pessoas pode evitar que perdas ocasionais de detecção em imagens isolados causem alternâncias rápidas na variável que representa o fluxo de pessoas.

Plataformas embarcadas de baixo custo (em comparação com equipamentos industriais como câmeras de alta definição ([Intelbras, 2026](#))), como as baseadas no microcontrolador *ESP32*, têm se tornado recorrentes em aplicações de Internet das Coisas, automação residencial e prototipagem acadêmica, devido à sua combinação de conectividade *Wi-Fi*, capacidade de processamento e suporte a bibliotecas de código aberto. O módulo *ESP32-CAM*, em particular, integra o *ESP32* a uma câmera *OV3660*, oferecendo uma solução compacta para captura e transmissão de imagens em projetos de visão computacional embarcada. Documentação e exemplos oficiais, como o firmware *CameraWebServer*, demonstram a viabilidade do uso desse módulo como câmera IP simples, acessível por protocolo *HTTP* ou *RTSP* ([Espressif Systems, 2024](#)).

A integração entre microcontroladores e técnicas de visão computacional permitiu a criação de um sistema semafórico inteligente capaz de reagir ao ambiente em tempo real. Enquanto os microcontroladores fornecem a infraestrutura física para aquisição de dados, comunicação e execução determinística da lógica de controle, a visão computacional amplia suas capacidades por meio da interpretação automática de imagens, identificando pedestres e veículos com apoio de modelos modernos.

2.3 Controladores lógicos programáveis e OpenPLC

Controladores Lógicos Programáveis (*CLPs*) são dispositivos eletrônicos projetados para executar, de forma determinística (seguindo regras ou algoritmos precisos onde a mesma entrada sempre produz o mesmo resultado), tarefas de automação industrial, substituindo lógicas por relés e temporizadores eletromecânicos por soluções programáveis (BOLTON, 2015). Esses equipamentos são amplamente utilizados em processos industriais, sistemas de energia, automação predial e, também, em controladores de tráfego, pela sua confiabilidade, capacidade de operação contínua e suporte a linguagens padronizadas pela norma *IEC 61131-3*. Essa norma define cinco linguagens principais, entre as quais se destacam *Ladder Diagram (LD)* e *Structured Text (ST)*, frequentemente adotadas para o desenvolvimento de lógicas de controle sequenciais e temporizadas (International Electrotechnical Commission, 2013; BOLTON, 2015).

No contexto de pesquisa acadêmica e prototipagem, a utilização de *CLPs* industriais pode se tornar inviável devido ao custo de aquisição e às restrições de acesso a equipamentos usados em campo. Desse modo, surge o interesse em plataformas abertas como o *OpenPLC*, que implementa um runtime compatível com *IEC 61131-3* em diferentes arquiteturas de hardware, incluindo microcontroladores como o *ESP32* (Autonomylogic, 2024; OpenPLC Project, 2018). O *OpenPLC* é composto por dois componentes principais: o Editor, utilizado para desenvolvimento de programas em linguagens padronizadas, e o *Runtime*, responsável por executar esses programas no dispositivo alvo. Essa abordagem permite transformar um *ESP32* em um controlador programável funcional, capaz de tornar variáveis internas visíveis por meio de protocolos industriais, como *Modbus/TCP*, e manipular entradas e saídas digitais conectadas a dispositivos físicos, como *LEDs* ou relés.

Documentos sobre *CLPs* destacam a importância de estruturas de controle robustas, incluindo o uso de temporizadores, estados bem definidos e mecanismos à prova de falhas para lidar com falhas de sensores ou de comunicação (BOLTON, 2015). Em sistemas de tráfego, esses princípios equivalem a modos de operação de segurança, como o acionamento de planos fixos, piscante amarelo ou tudo vermelho em caso de falha de detecção ou comunicação, alinhando-se a normas e boas práticas descritas em *NTCIP 1202* e documentos técnicos correlatos (NTCIP, 2019). Ao adotar o *OpenPLC* em conjunto com um *ESP32*, este trabalho reproduz em ambiente de laboratório a estrutura conceitual de um *CLP* semafórico real, mantendo a lógica de estados e temporizações dentro de um ambiente determinístico, mas com maior acessibilidade econômica e flexibilidade de desenvolvimento.

A integração entre microcontroladores e técnicas de visão computacional constitui o eixo central do projeto, permitindo a criação de um sistema semafórico inteligente capaz de reagir ao ambiente em tempo real. Enquanto os microcontroladores, representados aqui pelos módulos *ESP32* e *ESP32-CAM*, fornecem a infraestrutura física para aquisição de dados, comunicação e execução determinística da lógica de controle, a visão computacional

amplia suas capacidades por meio da interpretação automática de imagens, identificando pedestres com apoio de modelos modernos como *YOLO*. Essa união possibilita que dispositivos normalmente limitados a entradas digitais simples sejam integrados a sistemas de percepção avançada, aproximando a arquitetura laboratorial do funcionamento de sistemas urbanos inteligentes e evidenciando o potencial da combinação entre hardware embarcado e algoritmos de *IA* (inteligência artificial) para aplicações de mobilidade.

2.4 Comunicação industrial e protocolo Modbus/TCP

A integração entre sistemas de visão computacional e controladores programáveis, discutidos anteriormente, exige o uso de protocolos de comunicação padronizados, que permitam a troca confiável de dados em redes industriais ou redes *IP* convencionais. Entre esses protocolos, o *Modbus* se destaca por sua simplicidade, ampla adoção e disponibilidade em inúmeras plataformas e bibliotecas, tanto em versões seriais (*Modbus/RTU*) quanto em redes *TCP/IP* (*Modbus/TCP*) ([Modbus Organization, 2012](#); [Autonomylogic, 2024](#)).

O *Modbus/TCP* encapsula o protocolo Modbus em pacotes *TCP*, permitindo que clientes e servidores troquem informações em uma rede *Ethernet* padrão. Em aplicações típicas, um servidor *Modbus*, geralmente um *CLP*, expõe tabelas de registradores e bobinas que representam variáveis internas, enquanto um cliente, como um sistema supervisório, realiza leituras e escritas nesses endereços. A simplicidade do modelo, baseado em leitura e escrita de inteiros e bits, torna o protocolo especialmente adequado para aplicações acadêmicas e para a integração entre softwares de alto nível e controladores dedicados.

No contexto deste trabalho, o *runtime* (ambiente de execução) do *OpenPLC* em um *ESP32* assume o papel de servidor *Modbus/TCP*, expondo variáveis internas como contatos (instruções de entrada na linguagem *Ladder*), enquanto o programa que executa a visão computacional atua como cliente, utilizando a biblioteca *pymodbus* para estabelecer conexões, escrever estados de variáveis e, se necessário, ler estados de saída (bobinas) para fins de monitoramento ([Autonomylogic, 2024](#); [OpenPLC Project, 2018](#)). Esse arranjo replica, em escala reduzida, a arquitetura de sistemas reais em que módulos de detecção, unidades de processamento ou centrais de tráfego comunicam-se com controladores de campo através de protocolos padronizados, muitas vezes também baseados em perfis definidos por normas como *NTCIP 1202* ([NTCIP, 2019](#)).

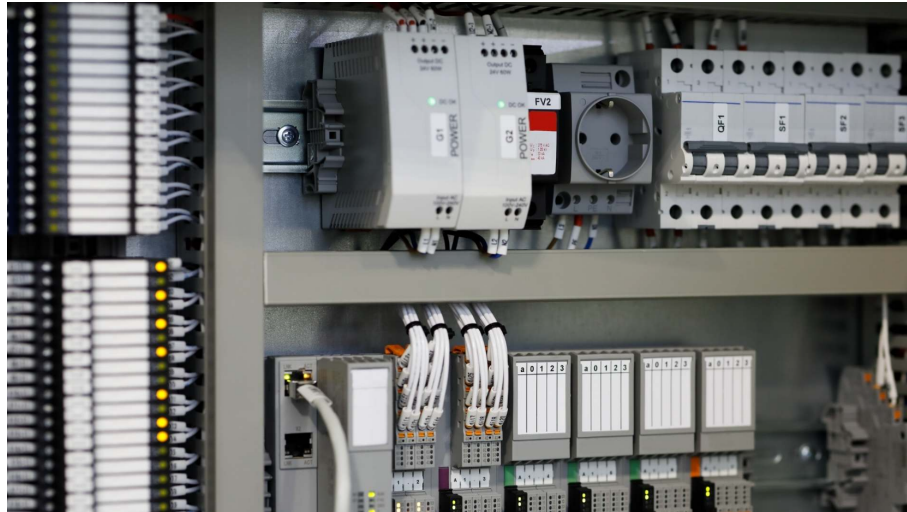


Figura 4 – Exemplo de um CLP industrial. Fonte: (CIM automação, 2017)

A adoção de *Modbus/TCP* na arquitetura proposta traz ainda um benefício, onde caso o protótipo evolua para a utilização de um *CLP* industrial semafórico que suporte esse protocolo, a camada de software de visão computacional pode ser mantida praticamente inalterada, exigindo apenas o remapeamento de endereços das variáveis, reforçando o caráter modular e escalável do projeto. Um exemplo de *CLP* industrial pode ser visualizado na Figura 4.

Os elementos apresentados ao longo desta fundamentação convergem para a construção de uma arquitetura distribuída típica de plataformas embarcadas, na qual sensores, unidades de processamento e controladores de campo atuam de forma integrada. Do ponto de vista teórico, a separação entre percepção (visão computacional), processamento lógico (agregação de detecções em variáveis binárias) e controle (*CLP* semafórico em *Ladder*) está em consonância com a literatura de automação, que recomenda camadas distintas para evitar acoplamento excessivo e facilitar manutenção, substituição de componentes e diagnóstico de falhas (GONZALEZ; WOODS, 2018; BOLTON, 2015; NTCIP, 2019).

3 METODOLOGIA

O capítulo a seguir foi estruturado a fim de demonstrar a concepção do projeto, através de tópicos que seguem toda a linha de raciocínio utilizada no decorrer do desenvolvimento, desde o estudo de caso ao desenvolvimento da integração entre os módulos que o compõe, de modo a, ao final do projeto, materializar um protótipo fiel, em escala de laboratório, de um sistema semafórico inteligente capaz de reagir à presença de pedestres detectados por visão computacional.

3.1 Estudo de Caso

O estudo de caso proposto tem como referência o funcionamento dos sistemas semafóricos na cidade de São Bernardo do Campo, Estado de São Paulo, inserida em um contexto de mobilidade urbana típico de regiões metropolitanas, com elevado fluxo veicular e de pedestres. Como não foram encontradas especificações públicas detalhadas sobre os controladores utilizados especificamente nesse município, optou-se por tomar como base técnica os manuais e documentos da Companhia de Engenharia de Tráfego de São Paulo, que descrevem critérios de programação semafórica, conceitos de tempos mínimos, entreverdes e operação de pedestres, além de diretrizes presentes no Manual Brasileiro de Sinalização de Trânsito ([CET – Companhia de Engenharia de Tráfego, 2026](#); [Brasil. Departamento Nacional de Trânsito, 2014](#)).

Com base em observações e em relatos técnicos, é pressuposto que a maior parte dos sistemas semafóricos do cenário estudado opera com planos de tempo fixo, sem atuação direta de sensores de presença ou demanda. Esse modelo tende a gerar tempos de espera desnecessários em períodos de baixo fluxo, bem como pode se mostrar insuficiente em horários de pico, quando a gestão de tráfego recorre a agentes de campo para intervenção manual, evidenciando a ausência de mecanismos mais bem adaptados ([CET – Companhia de Engenharia de Tráfego, 2014](#)).

Em comparação com a botoeira semafórica convencional, conforme Figura 5, a principal diferença da proposta desenvolvida está na forma de identificação da demanda de travessia. A botoeira é um dispositivo destinado a detectar a solicitação de pedestres e depende do acionamento manual do usuário para que o controlador implemente o tempo de travessia correspondente ([Brasil. Departamento Nacional de Trânsito, 2014](#)). Esse mecanismo apresenta como vantagens a simplicidade construtiva, a facilidade de integração com controladores semafóricos e a elevada previsibilidade de funcionamento. Entretanto, sua principal limitação é justamente depender da iniciativa do pedestre, caso o usuário não pressione o dispositivo, a demanda não é registrada. Além disso, a necessidade de deslocamento até a botoeira pode ser inconveniente em determinadas

situações, especialmente quando o pedestre não identifica facilmente o dispositivo, quando há grande fluxo de pessoas, quando o pedestre não apresenta mãos livres para uso ou por questões sanitárias, como na pandemia de *COVID-19*. A própria necessidade de sinalização educativa para indicar que o pedestre deve acionar a botoeira evidencia essa dependência da interação humana (Brasil. Departamento Nacional de Trânsito, 2014). Para botoeiras sonoras, essa interação pode envolver ainda procedimentos específicos, como a necessidade de pressionar o botão por pelo menos três segundos para ativar determinadas funções de acessibilidade (CONTRAN, 2022).



Figura 5 – Imagem de uma botoeira semafórica. Fonte: (Prefeitura de Joaçaba, 2022)

Este projeto propõe um sistema que represente a lógica de um controlador semafórico real, complementado por um subsistema de detecção automática de presença de pedestres por meio de visão computacional. Em vez de estimar fluxo ou demanda de forma contínua, como o projeto citado anteriormente (XIMENES et al., 2024), o protótipo planejado atua com variáveis binárias de presença, como as botoeiras semafóricas, porém de modo a tornar a operação mais dinâmica e transparente ao usuário (NTCIP, 2019). Serão adotados dois modos básicos de temporização: um ciclo padrão, que se trata da sequência convencional dos semáforos de tempo fixo, e um ciclo veicular, no qual o verde para veículos é mantido enquanto não houver pedestres aguardando, retornando ao ciclo padrão quando a permanência de pedestres for detectada por um intervalo mínimo.

Essa estrutura tem como o objetivo evidenciar a separação entre o nível de detecção,

associado à camada de visão computacional, e o nível de decisão, atribuído ao controlador semafórico. Tal separação é coerente com padrões internacionais, como os objetos definidos na família *NTCIP 1202* para controladores atuados, nos quais a lógica de controle consome variáveis abstratas de detecção, sem se vincular diretamente ao modo como essas variáveis são obtidas (NTCIP, 2019).

3.2 Projeção da Arquitetura Global do Sistema

A arquitetura para o sistema semafórico inteligente é modular e subdividida em três blocos: sistema visual, sistema de processamento e controlador semafórico. Essa divisão garante maior clareza na organização do projeto, facilita a reutilização de componentes e permite a evolução independente de cada subsistema sem a necessidade de reestruturação completa do sistema.



Figura 6 – 1048 Av. Piraporinha, São Bernardo do Campo. Fonte: Google Street View.

O cenário escolhido para o desenvolvimento do protótipo corresponde a uma via de mão única com travessia de pedestres, composta por um semáforo veicular e uma faixa para pedestres, conforme ilustrado na Figura 6. Naturalmente, duas câmeras seriam necessárias, uma para cada lado da via, porém, para fins de simplificação experimental, foi feita a utilização de apenas um módulo de sistema visual, responsável pela detecção dos pedestres. Em uma implementação real o número de módulos poderia ser maior, variando conforme a necessidade e complexidade do ambiente em questão.



Figura 7 – Arquitetura global do sistema. Fonte: Autoria própria.

O sistema visual é responsável pela aquisição e transmissão das imagens da cena. Para essa finalidade foi empregada uma placa *ESP32-CAM* equipada com sensor de imagem e interface de comunicação *Wi-Fi*, permitindo a captura de vídeo e sua transmissão em tempo real através da rede local do sistema (Espressif Systems, 2024). No protótipo implementado, a *ESP32-CAM* foi configurada para operar como uma câmera *IP*, transmitindo continuamente um fluxo de vídeo no formato *MJPEG* por meio de um servidor *HTTP* embarcado.

Idealmente, esse módulo deve ser instalado fisicamente em posição elevada e orientado de modo a captar a área da faixa de pedestres. O dispositivo realiza exclusivamente a captura e transmissão das imagens, sem executar processamento local de visão computacional e delegando essa tarefa ao módulo de processamento.

O módulo de processamento corresponde à unidade computacional responsável pela análise das imagens capturadas e pela geração de sinais de controle para o controlador semafórico. Nesse projeto, essa unidade consiste em um computador no qual foram implementados algoritmos de visão computacional em linguagem Python utilizando as bibliotecas *OpenCV* e *Ultralytics YOLO* (Ultralytics, 2024; SZELISKI, 2022). Esse módulo recebe o fluxo de vídeo da *ESP32-CAM*, processa os quadros capturados e determina, em tempo real, a presença de pedestres na área monitorada.

O controlador semafórico é implementado utilizando um microcontrolador *ESP32* executando o runtime do *OpenPLC*. Esse ambiente permite a programação da lógica de controle segundo a norma *IEC 61131-3*, em especial na linguagem Ladder, aproximando o protótipo de uma arquitetura típica de um *CLP* industrial (Autonomylogic, 2024). O controlador recebe variáveis lógicas provenientes do sistema de processamento por meio do protocolo *Modbus/TCP* e utiliza essas informações para alternar o ciclo de funcionamento do semáforo entre o modo intermitente, onde os veículos podem trafegar livremente, e o modo normal, onde o semáforo opera com seu ciclo padrão de luzes para veículos e pedestres, além de receber informações sobre a disponibilidade dos outros módulos para

implementação do modo de segurança, onde a luz amarela do semáforo veicular permanece piscando para informar a indisponibilidade do sistema.

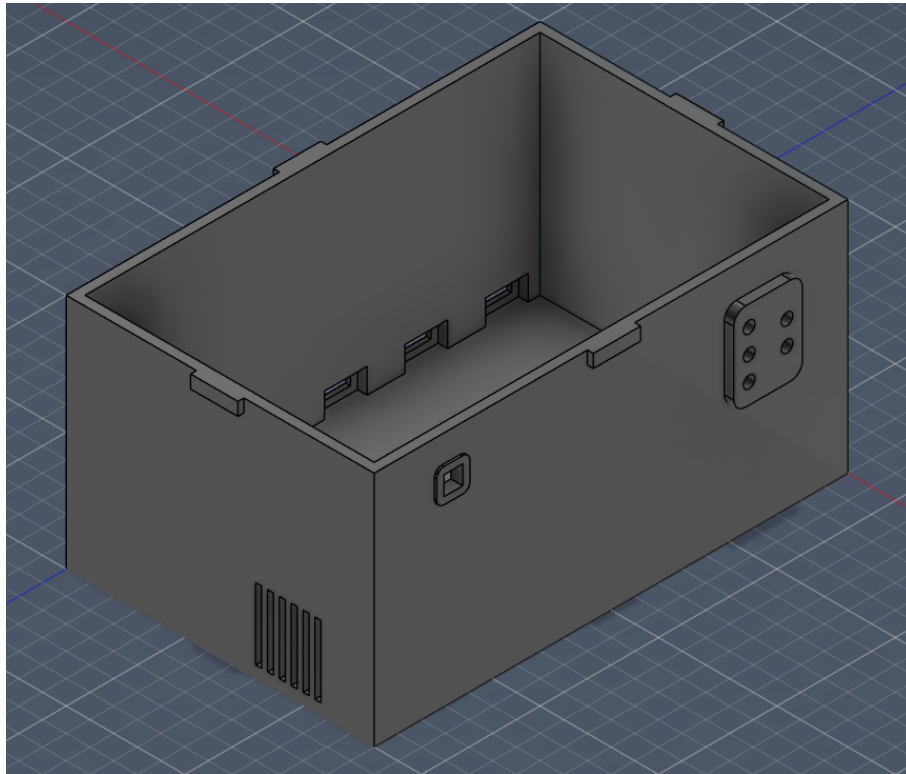


Figura 8 – Visão frontal do compartimento. Fonte: Autoria própria.

Para fins de prototipagem, os módulos de captura de imagem e de controlador semafórico foram unidos em um único compartimento. Este compartimento (Figura 8) tem a finalidade de abrigar todos os componentes eletrônicos sensíveis e garantir segurança no transporte, além de simplificar a implementação e os testes iniciais.

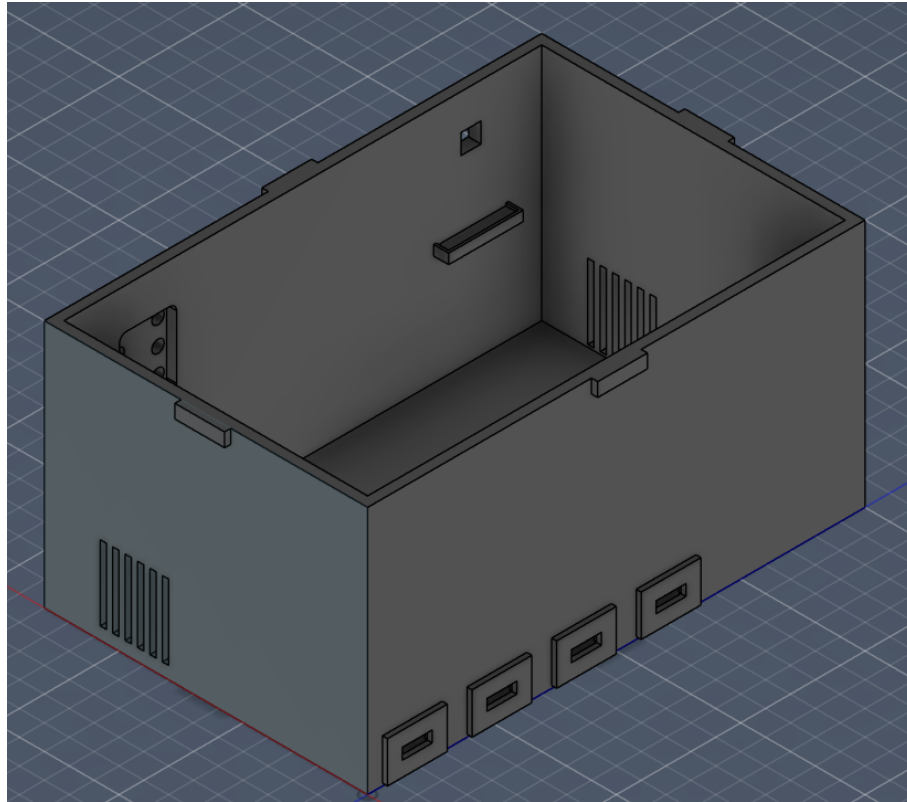


Figura 9 – Visão traseira do compartimento. Fonte: Autoria própria.

O compartimento foi projetado através do software *Autodesk Fusion*, um software profissional de *CAD*, *CAM*, *CAE* e *PCB* em nuvem que une design 3D, simulação, engenharia e fabricação em uma única plataforma, sendo amplamente utilizado para engenharia mecânica, design de produtos e prototipagem. Ele possui um alojamento interno para o *ESP32-CAM* com abertura para sua câmera (lado esquerdo frontal da Figura 8), além de furos para os *LEDs* que representam as luzes do semáforo. Também possui quatro saídas para alojamento das conexões *USB* que alimentam o *ESP32-CAM* e o *ESP32* do módulo do controlador semafórico, como é possível ver na Figura 9. Um paquímetro foi utilizado para obtenção das medidas exatas dos componentes.

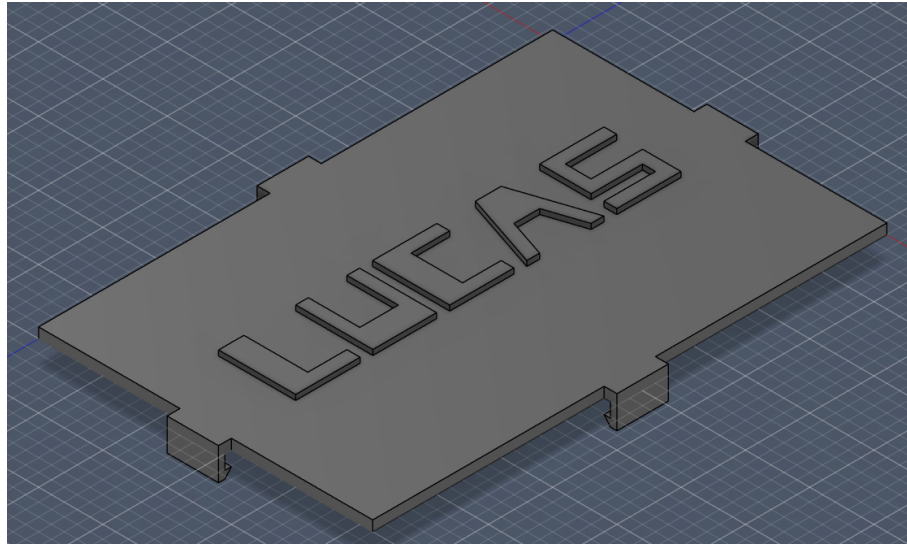


Figura 10 – Tampa do compartimento. Fonte: Autoria própria.

Também foi projetada uma tampa para fácil acesso e proteção aos componentes internos (Figura 10). Ambas as partes foram impressas utilizando uma impressora 3D. O compartimento também abrigará uma protoboard e todos os cabos utilizados no projeto.

3.3 Desenvolvimento do Sistema Visual

O sistema visual foi implementado utilizando um módulo *ESP32-CAM* baseado no microcontrolador *ESP32* e equipado com o sensor de imagem *OV3660*, o qual possui resolução máxima de 2 megapixels e suporte a compressão JPEG em hardware ([Espressif Systems, 2024](#)). Essa plataforma foi escolhida por apresentar baixo custo em comparação com outras câmeras IP de alta definição ([Intelbras, 2026](#)), integração com rede *Wi-Fi* e ampla disponibilidade de exemplos e documentação técnica.

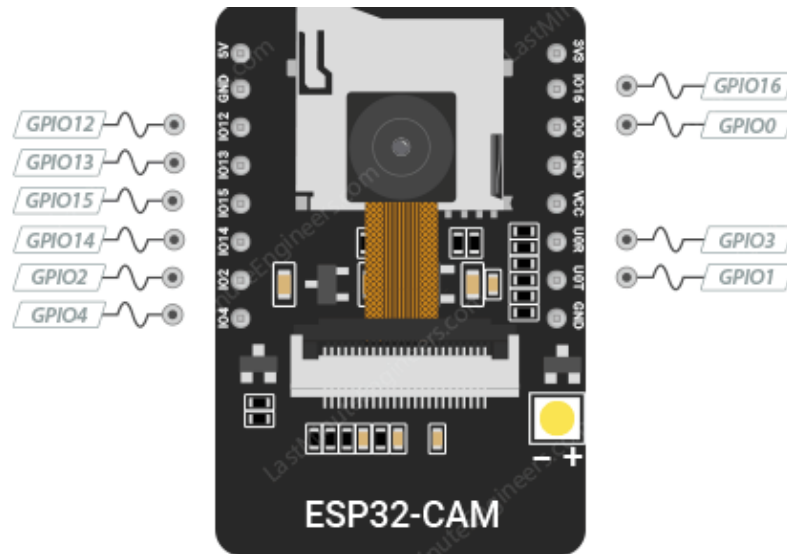


Figura 11 – Esp32-CAM Pinout. Fonte: (PRABHU, 2026a).

Para fins de prototipagem e de modo a simplificar o projeto inicial, a alimentação do módulo foi realizada por meio de uma fonte 5 V estabilizada, obtida através da saída *USB* do computador e de um cabo *USB* cuja extremidade foi adaptada para dois conectores jumpers, positivo e negativo, que foram conectados aos pinos 5 V e *GND* da placa, conforme pinos da Figura 11.



Figura 12 – ESP32-CAM (esquerdo) ao lado do Módulo Adaptador. Fonte: Autoria própria.

A programação da *ESP32-CAM* foi realizada utilizando o ambiente Arduino a partir do exemplo oficial fornecido pela *Espressif* para transmissão de vídeo, denominado *CameraWebServer*, que foi adaptado para atender às necessidades do projeto, conforme melhor abordado no Apêndice A. Esse exemplo pode ser obtido, no programa *Arduino IDE*, através da aba “*File>Examples>ESP32>Camera>CameraWebServer*” após a instalação do pacote “*esp32*” da *Espressif* na sessão “*BOARDS MANAGER*” do aplicativo. A *ESP32-CAM* foi conectada ao computador através do barramento serial via cabo *USB* e um Módulo Adaptador, conforme as imagens das Figuras 12 e 13. No programa *Arduino IDE* é necessário selecionar a placa utilizada e a porta *COM* na qual ela se encontra conectada. Inicialmente foi implementado um servidor *HTTP* embarcado capaz de transmitir continuamente o fluxo de vídeo capturado pela câmera no formato *MJPEG*. Essa transmissão ocorre através de uma rota *HTTP* dedicada, permitindo que o sistema de processamento acesse os quadros capturados pela câmera via rede local, atingindo resoluções a partir de 640x480 pixels.



Figura 13 – ESP32-CAM conectado ao Módulo Adaptador e ao barramento USB do computador. Fonte: Autoria própria.

3.4 Desenvolvimento do Sistema de Processamento

O sistema de processamento foi implementado em um computador executando sistema operacional *Windows* (onde qualquer outro sistema compatível com as tecnologias utilizadas poderia ser empregado), escolhido por sua estabilidade, disponibilidade de bibliotecas científicas e compatibilidade com ferramentas modernas de aprendizado de máquina (SZELISKI, 2022). A linguagem de programação adotada foi *Python 3*, amplamente utilizada em aplicações de visão computacional e inteligência artificial. É através das variáveis lógicas emitidas pelo algoritmo deste módulo que o controlador decide qual modo de operação (normal, intermitente ou de segurança) adotará.

Para a captura do fluxo de vídeo transmitido pela *ESP32-CAM* foram utilizadas as bibliotecas *OpenCV*, *numpy* e *requests*, responsáveis por receber e decodificar o stream *MJPEG* disponibilizado pelo servidor *HTTP* embarcado na câmera. Durante os testes iniciais foi observado que a utilização direta do mecanismo padrão de captura de vídeo do *OpenCV* poderia introduzir atrasos e buffers excessivos no processamento de streams *MJPEG*, gerando problemas na etapa de detecção e na exibição do vídeo. Para resolver esse problema, foi implementado um mecanismo de leitura de fluxo otimizado, utilizando requi-

sições *HTTP* contínuas e decodificação manual dos quadros *JPEG* recebidos, garantindo que o sistema de processamento utilize sempre o quadro mais recente disponível.

Após a captura do quadro de vídeo, o sistema aplica uma etapa de pré-processamento utilizando a biblioteca *OpenCV*. Entre as operações realizadas estão a inversão vertical da imagem, redimensionamento para resolução padronizada e ajustes de brilho e contraste (conforme Apêndice B), com o objetivo de melhorar a qualidade visual das imagens antes da etapa de detecção de objetos. Os valores foram escolhidos com base em pequenos ajustes e observações realizados durante as etapas de experimentação do algoritmo.

A detecção de pedestres foi implementada utilizando o modelo *YOLOv8n* disponibilizado pela biblioteca Ultralytics (Ultralytics, 2024). Esse modelo foi escolhido por apresentar boa relação entre desempenho computacional e precisão de detecção, sendo adequado para aplicações em tempo real em computadores de capacidade moderada, onde se espera um desempenho compatível com processamento próximo ao tempo real. O modelo é executado sobre cada quadro capturado e retorna as regiões da imagem que contêm objetos detectados pertencentes às classes definidas na base de dados *COCO*.

Para este projeto, a detecção foi restrita à classe correspondente a pessoas, reduzindo o custo computacional do processo e aumentando a eficiência da inferência. Sempre que ao menos uma pessoa é detectada no quadro analisado, o sistema interpreta esse evento como presença de pedestres na área monitorada.

Durante os testes, o programa apresentava dificuldades para realizar a detecção, o que gerava oscilações e travamentos do módulo do controlador semafórico. A fim de evitar essas oscilações indesejadas na detecção decorrentes de ruídos ou falsas detecções momentâneas, foi implementado um mecanismo de histerese temporal. Nesse mecanismo, a presença de pedestres só é confirmada após um número mínimo de quadros consecutivos contendo detecção positiva, enquanto a ausência de pedestres só é confirmada após um número mínimo de quadros consecutivos sem detecção, números escolhidos de forma arbitrária conforme testes realizados durante a etapa de experimentação e desenvolvimento. Essa abordagem é recomendada na literatura de visão computacional para sistemas de decisão em tempo real (SZELISKI, 2022). Toda a lógica implementada nesta etapa pode ser consultada no Apêndice B.

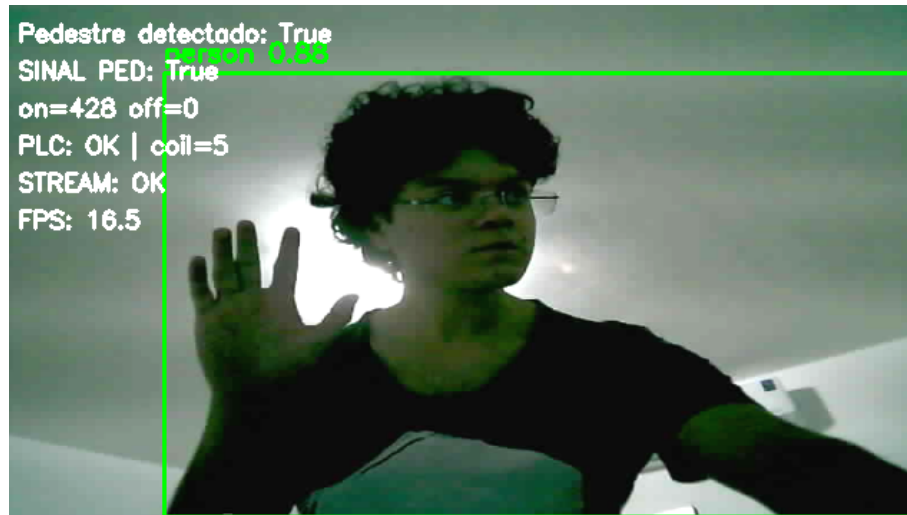


Figura 14 – Exemplo de detecção de uma pessoa através do algoritmo utilizado. Fonte: Autoria própria.

Após a detecção e filtragem temporal, o sistema gera uma variável lógica indicando a presença ou ausência de pedestres. Essa variável é transmitida ao controlador semafórico por meio do protocolo *Modbus/TCP*, utilizando uma biblioteca cliente implementada em *Python*. O computador atua como cliente *Modbus* e escreve o valor da variável em uma bobina previamente definida no controlador *OpenPLC*.

Além da variável que indica a presença de pedestres, foi implementado também um mecanismo de *heartbeat*, que consiste na alternância periódica de uma segunda variável lógica (*heartbeat*) enviada ao controlador. Esse mecanismo permite que o controlador detecte falhas de comunicação ou travamentos no sistema de processamento e, através da lógica *Ladder*, acione o modo de segurança. O mesmo é feito para o monitoramento da disponibilidade da câmera, onde o algoritmo deste módulo envia constantemente, para o controlador, os dados de *status* de conexão da câmera.

3.5 Desenvolvimento do Controlador Semafórico

O controlador semafórico foi implementado utilizando um microcontrolador *ESP32*, conforme Figura 15, executando o runtime *OpenPLC*, permitindo a programação da lógica de controle na linguagem *Ladder* conforme especificado pela norma *IEC 61131-3* (Autonomylogic, 2024). Essa abordagem permite reproduzir o comportamento de um controlador lógico programável típico utilizado em sistemas de automação industrial.

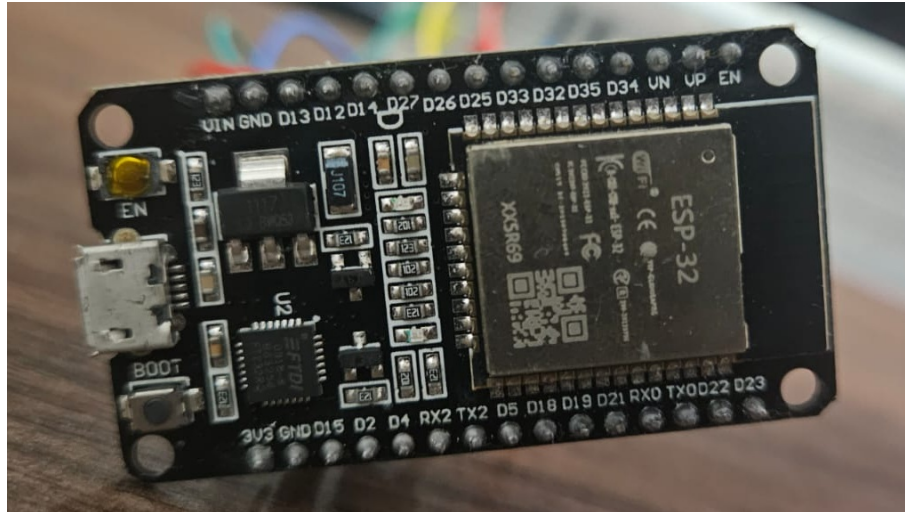


Figura 15 – ESP32 utilizado como controlador semafórico. Fonte: Autoria própria.

O procedimento de configuração do *ESP32* foi feito realizando a conexão entre ele e o computador através do barramento *USB*, que também fornece energia para o componente, onde o programa *OpenPLC Editor* foi utilizado para escrever a lógica de controle dos *LEDs*, que representam as lâmpadas de um semáforo. O esquema de pinos do *ESP32* utilizado pode ser visto na Figura 16.

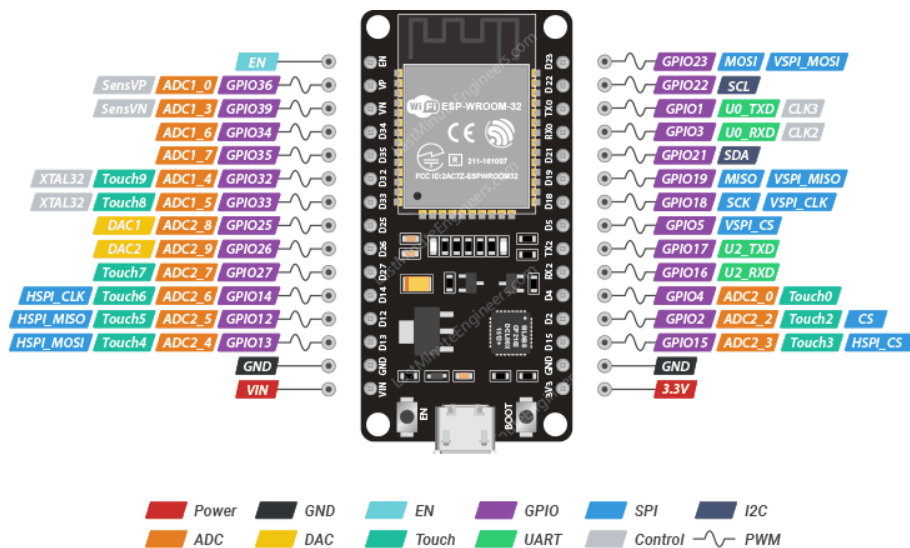


Figura 16 – ESP32 Pinout. Fonte: (PRABHU, 2026b).

Como demonstrado na Figura 17, inicialmente foi feita a configuração do tipo de dispositivo no programa. A opção “*ESP32 Generic [x.x.x]*” foi escolhida por reconhecer a maioria dos modelos sem problemas.

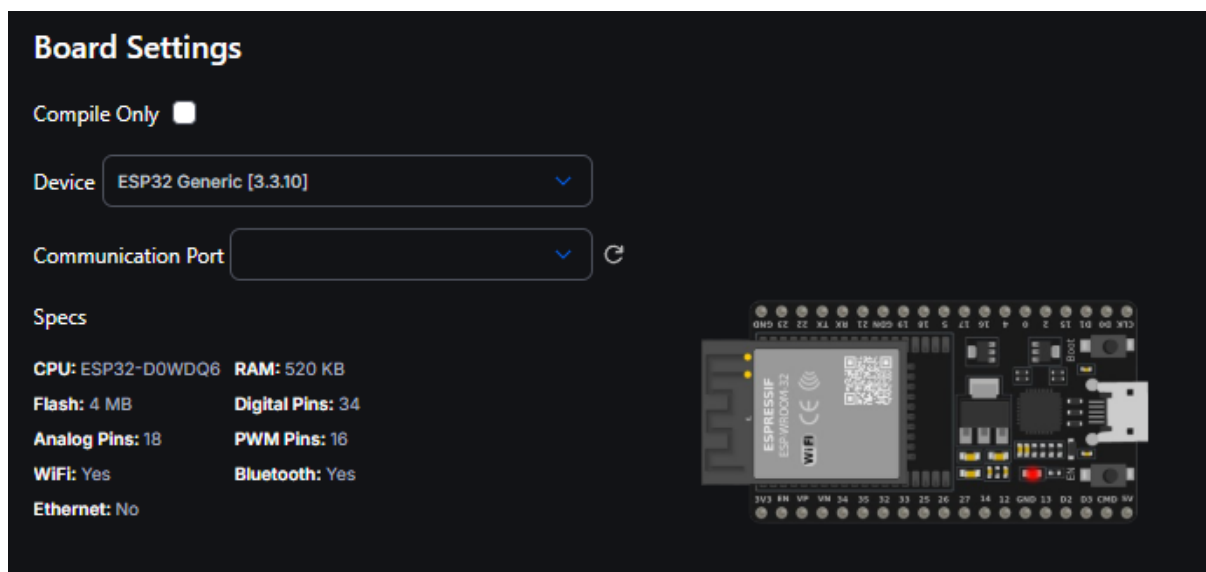


Figura 17 – Configuração do ESP32 no software OpenPLC Editor. Fonte: Autoria própria.

Além disso, conforme Figura 18, foi necessário também fornecer as informações de comunicação e o protocolo utilizado que, no caso deste projeto, se trata do *Modbus/TCP*.

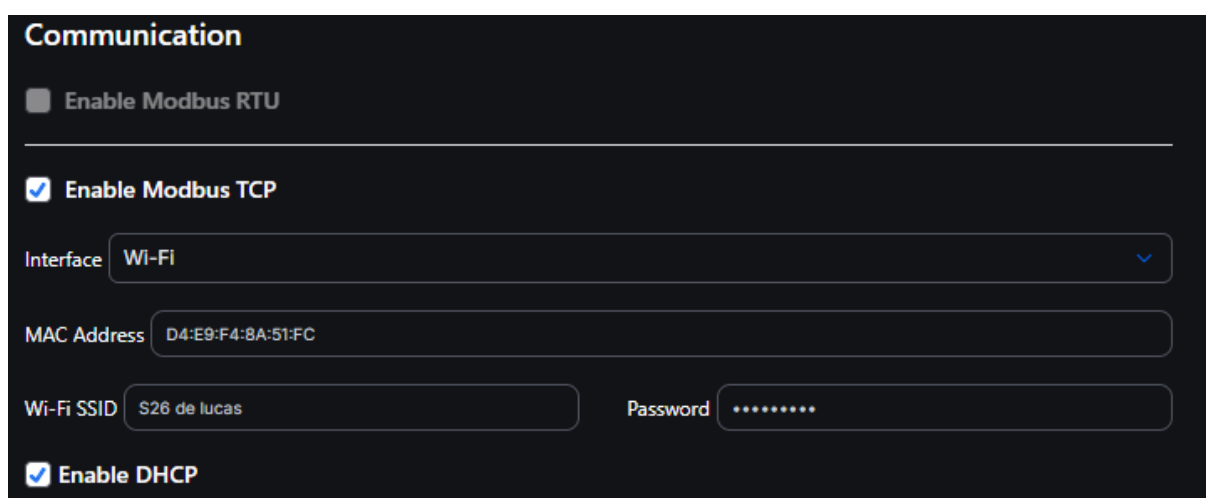


Figura 18 – Configuração da comunicação do ESP32 no software OpenPLC Editor. Fonte: Autoria própria.

Em seguida, foi feito o endereçamento dos pinos físicos do *ESP32* no software, para que fossem reconhecidos como variáveis, conforme pode ser visto pela Figura 19. Os pinos 2, 4 e 5 foram utilizados para, respectivamente, as cores verde, amarelo e vermelho do semáforo veicular e os pinos 12 e 14 para, respectivamente, as cores verde e vermelho do semáforo de pedestre. Os endereços virtuais são definidos automaticamente pelo software.

Pin	Type	Address	Name
02	Digital Output	%QX0.0	verde
04	Digital Output	%QX0.1	amarelo
05	Digital Output	%QX0.2	vermelho
12	Digital Output	%QX0.3	verde_ped
14	Digital Output	%QX0.4	vermelho_ped
39	Digital Output	%QX0.5	tem_ped
36	Digital Output	%QX0.6	heartbeat
17	Digital Output	%QX0.7	cameraok

Figura 19 – Configuração dos pinos do ESP32 no software OpenPLC Editor. Fonte: Autoria própria.

Em seguida, conforme as Figuras 20, 21 e 22, foi feita a definição das variáveis que serão utilizadas na lógica *Ladder* de controle dos *LEDs*. São utilizadas cinco variáveis para cada *LED* mencionado anteriormente, além de dois tempos, um de dois segundos para a transição entre amarelo e outras cores, e um de cinco segundos para a duração das cores verde e vermelho (e para o tempo até o acionamento do sistema contra falhas), três temporizadores de retardo na energização (*TONs*) para controle do semáforo veicular (e mais um para o sistema contra falhas), e oito variáveis de memória para elaboração da lógica.

#	Name	Class	Type	Location	Initial Value
0	VERDE	Local	BOOL	%QX0.0	
1	VERMELHO	Local	BOOL	%QX0.2	
2	AMARELO	Local	BOOL	%QX0.1	
3	VERDE_PED	Local	BOOL	%QX0.3	
4	VERMELHO_PED	Local	BOOL	%QX0.4	
5	TEM_PED	Local	BOOL	%QX0.5	
6	DELAY2	Local	TIME		T#2s
7	DELAY5	Local	TIME		T#5s
8	M1	Local	BOOL		

Figura 20 – Configuração das variáveis no software OpenPLC Editor (parte 1). Fonte: Autoria própria.

Para permitir a integração com o sistema de processamento, foi criada uma variável lógica interna no programa *Ladder*, associada a uma bobina *Modbus*. Essa variável,

configurada na Figura 19 como pino 39, representa a presença de pedestres detectada pelo sistema de visão computacional. A escolha desse pino foi arbitrária. O mapeamento dessa variável para o endereço *Modbus* foi realizado por meio das configurações de hardware do *OpenPLC*, associando a variável a um endereço específico de bobina acessível via protocolo *Modbus/TCP*.

9	M2	Local	BOOL		
10	M3	Local	BOOL		
11	TON1	Local	TON		
12	TON2	Local	TON		
13	TON3	Local	TON		
14	v1	Local	BOOL		
15	v2	Local	BOOL		
16	v3	Local	BOOL		
17	v1p	Local	BOOL		
18	v3p	Local	BOOL		
19	CameraOk	Local	BOOL	%QX0.7	

Figura 21 – Configuração das variáveis no software OpenPLC Editor. (parte 2) Fonte: Autoria própria.

Aqui a lógica Ladder implementa dois modos principais de operação. O primeiro corresponde ao ciclo padrão do semáforo, no qual as fases de sinalização seguem uma sequência temporal fixa entre os estados de verde, amarelo e vermelho para os veículos e pedestres. O segundo modo corresponde ao ciclo intermitente, no qual o tempo de sinal verde para veículos pode ser estendido na ausência de pedestres detectados pelo sistema visual.

20	Heartbeat	Local	BOOL	%QX0.6	
21	HB_mudou	Local	BOOL		
22	TON_heartbeat	Local	TON		
23	R_TRIG0	Local	R_TRIG		
24	Emitte_falha	Local	BOOL		
25	F_TRIG0	Local	F_TRIG		
26	trava	Local	BOOL		
27	TON4	Local	TON		

Figura 22 – Configuração das variáveis no software OpenPLC Editor (parte 3). Fonte: Autoria própria.

Adicionalmente, foi previsto um mecanismo de supervisão baseado em um sinal de *heartbeat* (pulsos contínuos) e um sinal de monitoramento da conexão com a câmera enviados

pelo sistema de processamento, conforme descrito na seção anterior. Esse mecanismo permite que o controlador detecte eventuais falhas na comunicação ou no sistema de visão computacional, acionando o ciclo de alerta do semáforo em caso de perda de comunicação, onde a luz amarela permanece oscilando até que a conexão seja reestabelecida, em conformidade com recomendações de segurança para sistemas de controle de tráfego (NTCIP, 2019). Esses sinais são recebidos pelo controlador através dos pinos 36 e 17 (*heartbeat* e câmera, respectivamente), conforme Figura 19. A lógica é melhor abordada no Apêndice C.

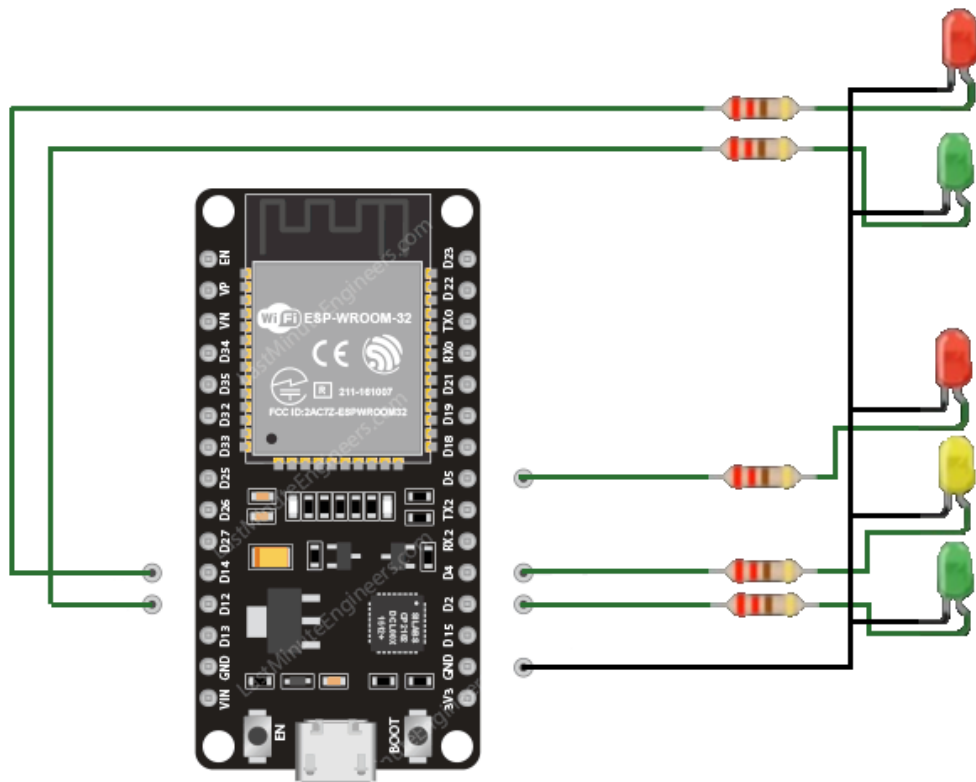


Figura 23 – Esquema de interligação entre os LEDs e o ESP32. Fonte: Autoria própria.

Fisicamente, os estados do semáforo foram representados por *LEDs* conectados às saídas digitais do *ESP32*, simulando as lâmpadas dos semáforos veiculares e de pedestres. Conforme Figura 23, as saídas D2, D4 e D5 foram utilizados para, respectivamente, as cores verde, amarelo e vermelho do semáforo veicular e os pinos D12 e D14 para, respectivamente, as cores verde e vermelho do semáforo de pedestre. Cada saída é conectada a um resistor de 220 Ohms e, posteriormente, ao pino *Ground* (*GND*) do *ESP32*, as conexões foram realizada com o auxílio de uma protoboard, ilustrada pela Figura 24.

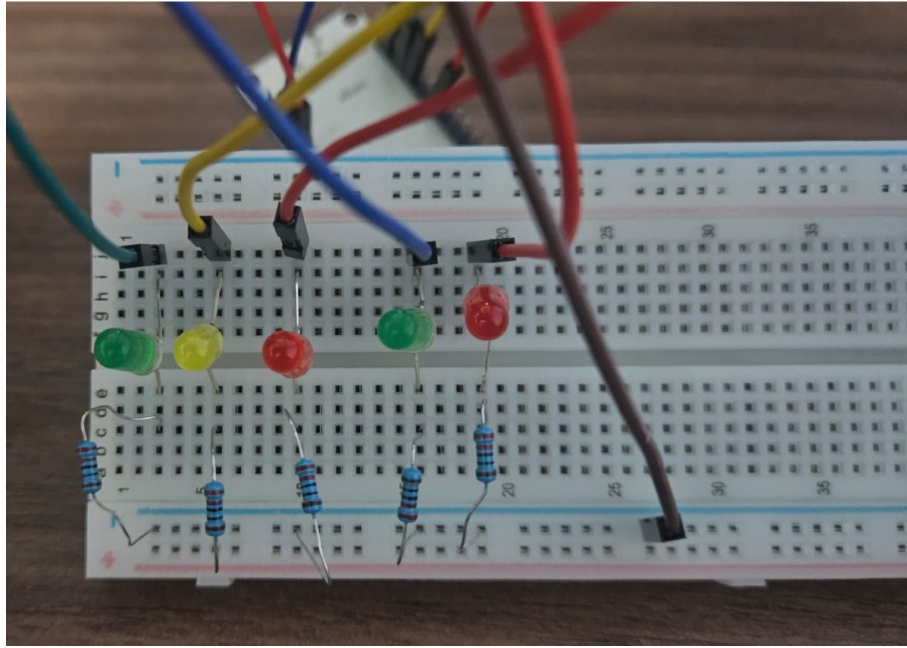


Figura 24 – Circuito semafórico para interligação entre os LEDs e o ESP32. Fonte: Autoria própria.

3.6 Integração entre os Módulos

A integração entre os módulos do sistema foi implementada como um fluxo contínuo de informação que parte da aquisição visual, passa pelo processamento computacional e culmina na execução da lógica de controle semafórico. A *ESP32-CAM* captura e transmite continuamente o fluxo de vídeo da cena por meio da rede *Wi-Fi* e utilizando protocolo *HTTP*. Esse fluxo é recebido pelo computador responsável pelo sistema de processamento, que executa os algoritmos de visão computacional e determina a presença de pedestres.

As variáveis lógicas geradas pelo sistema de processamento são então transmitidas ao controlador semafórico por meio do protocolo *Modbus/TCP*. O controlador *OpenPLC* recebe essas informações e as utiliza dentro da lógica *Ladder* para determinar qual fase do semáforo deve ser ativada em cada ciclo de varredura.

Essa arquitetura modular permite que sensores, unidades de processamento e controladores de campo operem de forma cooperativa através de protocolos de comunicação padronizados. Embora implementado em escala de protótipo, o sistema reproduz de forma conceitual a arquitetura de sistemas reais de controle de tráfego, nos quais técnicas de visão computacional e redes industriais são utilizadas para melhorar a eficiência do fluxo urbano sem comprometer os requisitos de segurança operacional (HARRISON; ABBOTT, 2015). O fluxograma ilustrado na Figura 25 exemplifica visualmente todo o processo de interação entre os módulos.

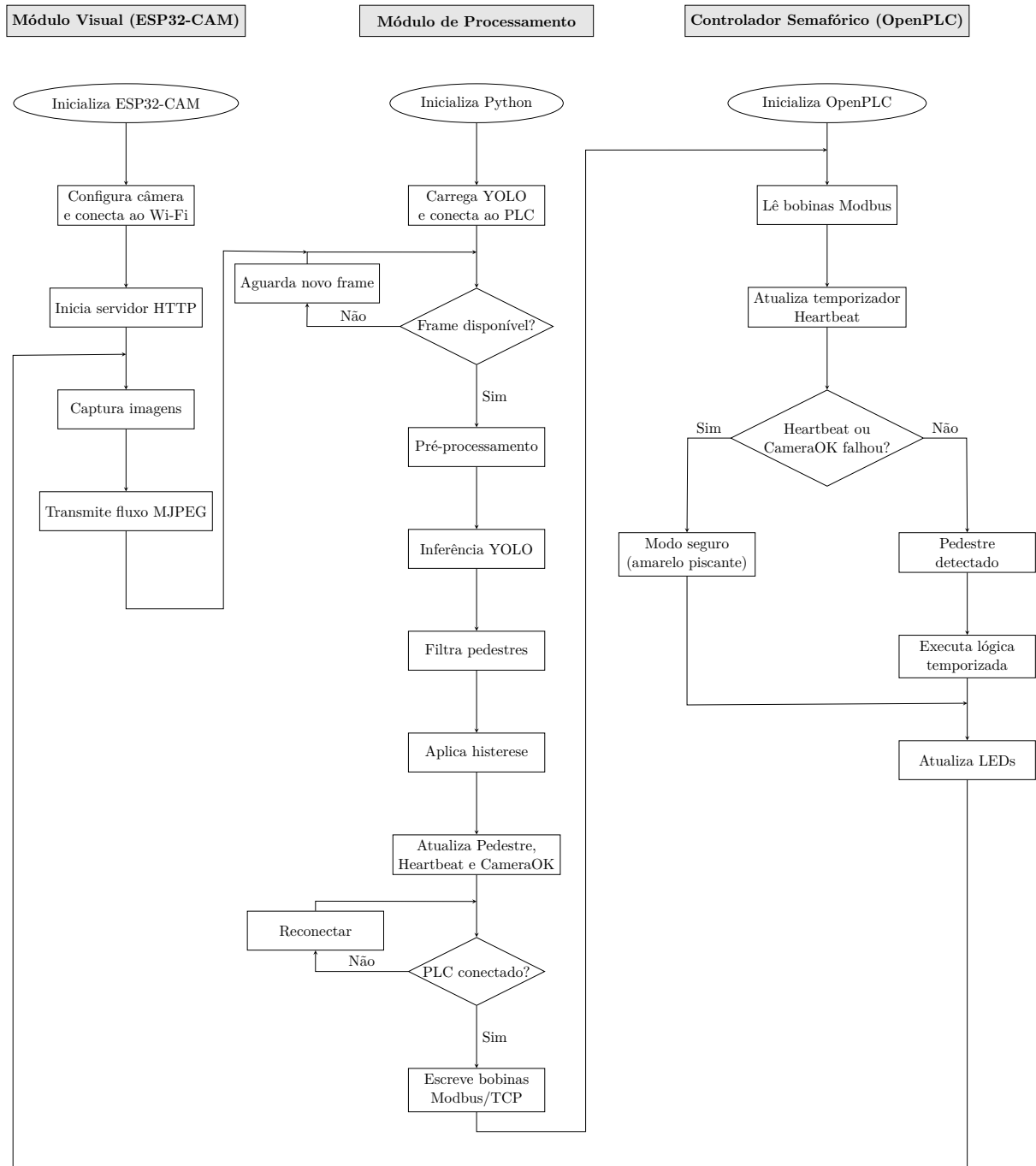


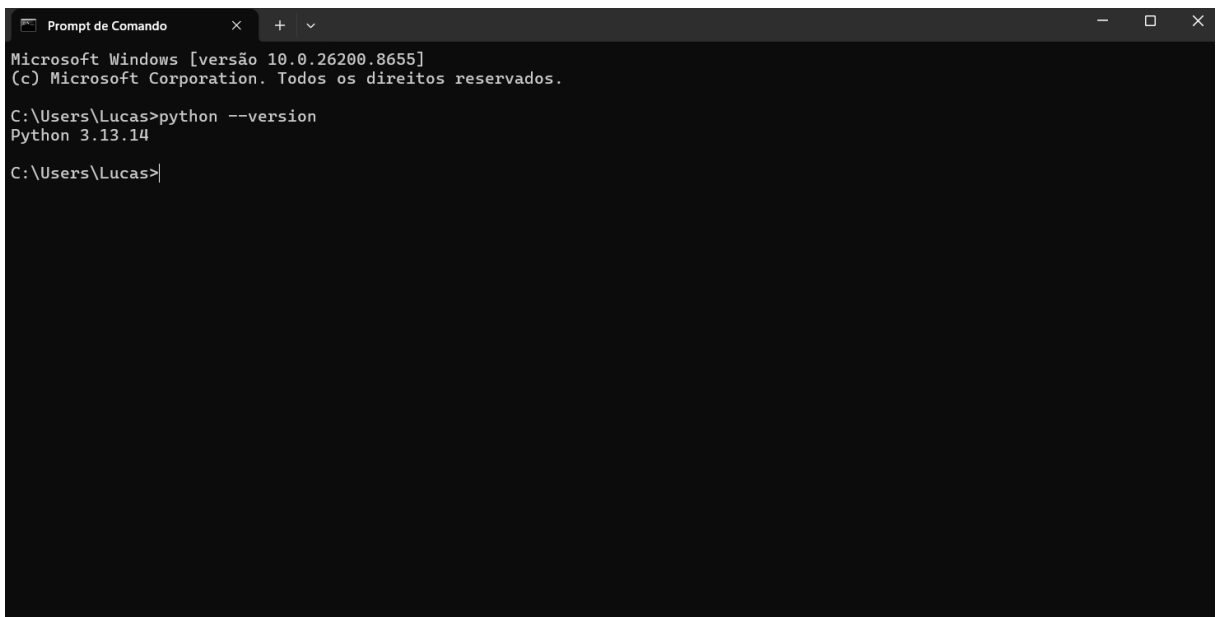
Figura 25 – Fluxograma do sistema semafórico inteligente. Fonte: Autoria própria.

3.7 Instalação e Execução do Programa de Processamento

A execução do módulo de processamento foi projetada para ser simples e independente de ambientes de desenvolvimento específicos, permitindo que o sistema seja instalado e executado diretamente pelo *Prompt de Comando (CMD)*. Embora o desenvolvimento do software tenha sido realizado utilizando o *Visual Studio Code (VS Code)*, sua utilização não é obrigatória para a execução do programa, uma vez que a *IDE* atua apenas como

ferramenta de edição e depuração. Dessa forma, qualquer computador que possua o interpretador *Python* e as bibliotecas necessárias instaladas é capaz de executar o sistema normalmente.

O primeiro passo consiste na instalação do interpretador *Python*, o que pode ser realizado pelo site oficial do desenvolvedor. Durante o processo de instalação, é importante habilitar a opção que adiciona automaticamente o *Python* às variáveis de ambiente do sistema (*Add Python to PATH*), permitindo sua utilização diretamente pelo terminal. Após a conclusão da instalação, recomenda-se verificar seu funcionamento por meio do comando `python --version`, que informa a versão instalada do interpretador, conforme Figura 26.



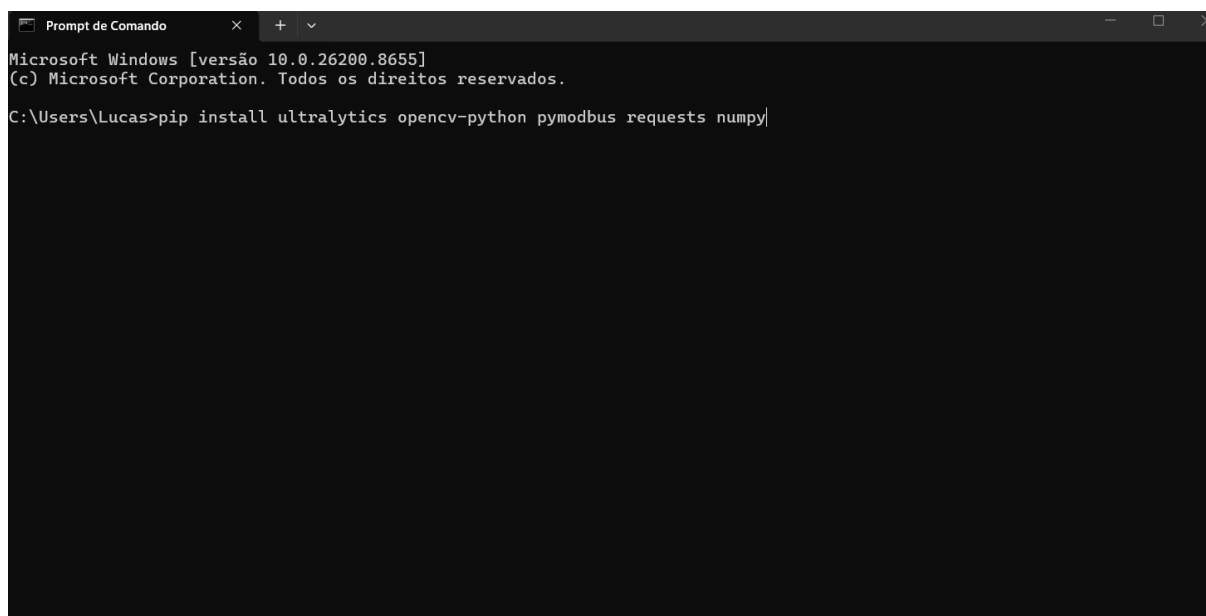
```
Prompt de Comando
Microsoft Windows [versão 10.0.26200.8655]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Lucas>python --version
Python 3.13.14

C:\Users\Lucas>
```

Figura 26 – Conferência da versão do python após instalação. Fonte: Autoria própria.

Em seguida, recomenda-se criar uma pasta destinada ao projeto e armazenar nela todos os arquivos necessários para a execução do sistema, incluindo o código-fonte em Python e o modelo de inteligência artificial utilizado pelo algoritmo *YOLOv8*. Embora não seja obrigatório, também é recomendada a criação de um ambiente virtual utilizando a ferramenta *venv*, recurso nativo do *Python* que permite isolar as bibliotecas utilizadas pelo projeto das demais bibliotecas instaladas no computador. Após a criação do ambiente virtual, ele deve ser ativado antes da instalação das dependências do sistema.



```
Prompt de Comando
Microsoft Windows [versão 10.0.26200.8655]
(c) Microsoft Corporation. Todos os direitos reservados.
C:\Users\Lucas>pip install ultralytics opencv-python pymodbus requests numpy|
```

Figura 27 – Instalação das bibliotecas necessárias para o programa no terminal do Windows.
Fonte: Autoria própria.

As bibliotecas necessárias podem ser instaladas diretamente pelo gerenciador de pacotes *pip*, distribuído juntamente com o *Python*, sendo elas as bibliotecas *OpenCV*, *NumPy*, *Requests*, *PyModbus* e *Ultralytics*, conforme Figura 27. Durante a instalação, o próprio gerenciador de pacotes realiza automaticamente o download das dependências secundárias, como *PyTorch*, *TorchVision* e demais bibliotecas necessárias ao funcionamento do sistema, simplificando significativamente o processo de configuração. Após a conclusão da instalação das dependências, a execução do sistema pode ser realizada diretamente pelo *Prompt de Comando* por meio da chamada do interpretador *Python* seguida do arquivo principal do projeto, conforme Figura 28.

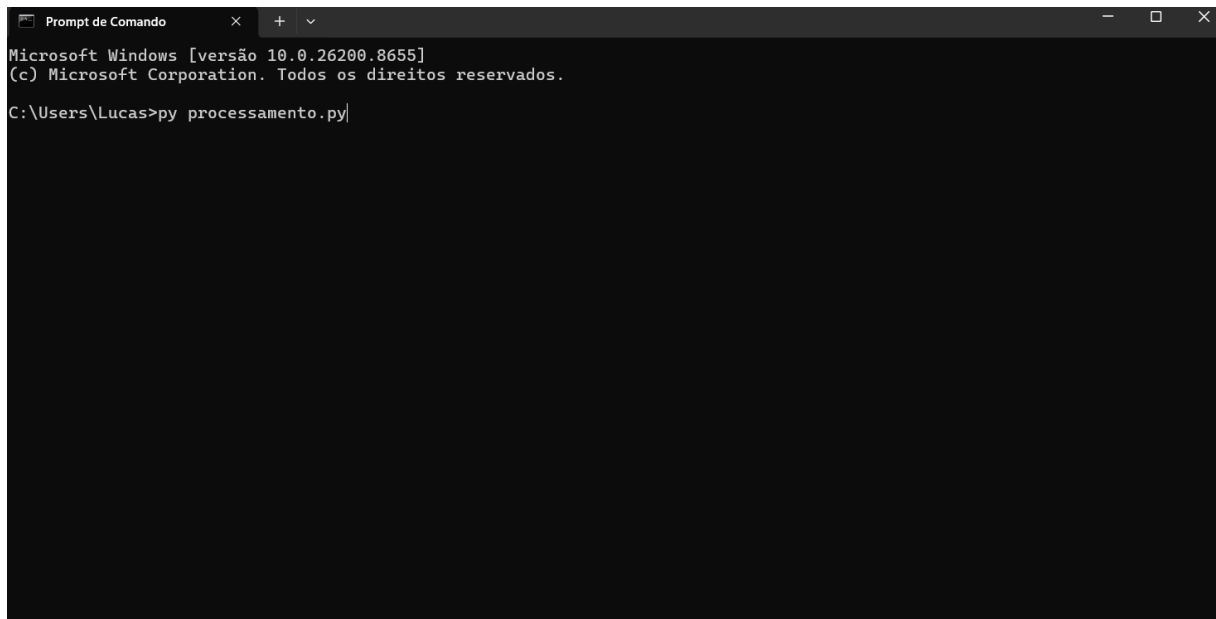


Figura 28 – Iniciando programa através do terminal do windows. Fonte: Autoria própria.

4 RESULTADOS E DISCUSSÃO

O desenvolvimento do protótipo permitiu validar com sucesso, embora com algumas ressalvas, a proposta de integração entre técnicas de visão computacional e controladores lógicos programáveis em um cenário representativo de controle semafórico inteligente. Ao longo do projeto foram implementados os três principais módulos da arquitetura proposta, sendo eles a aquisição de imagens por meio da *ESP32-CAM*, processamento utilizando visão computacional em *Python* e atuação do controlador desenvolvido em linguagem *Ladder*. A comunicação entre esses módulos através da rede local utilizando o protocolo *Modbus/TCP* foi bem-sucedida, demonstrando que diferentes plataformas de hardware e software podem operar de maneira integrada em aplicações de automação.



Figura 29 – Vista frontal do compartimento que abriga o módulos de visão e o controlador.
Fonte: Autoria própria.

O protótipo final pode ser observado nas Figuras 29 e 30. Feito utilizando impressão 3D, ele apresentou uma estrutura bastante rígida, inclusive para a tampa, devido ao plástico *ABS* (polímero termoplástico rígido, leve e de alto impacto) utilizado e à espessura elevada, o que dificultou o encaixe e desencaixe entre as partes. As medidas da base de apoio do *ESP32-CAM* não ficaram adequadas, resultando em pouco espaço útil para seu encaixe e na necessidade de sua remoção para que a câmera fosse corretamente posicionada no furo destinado a ela. Os furos para a câmera, *LEDs* e saídas *USB*, no entanto, apresentaram dimensões adequadas permitiram um encaixe firme.

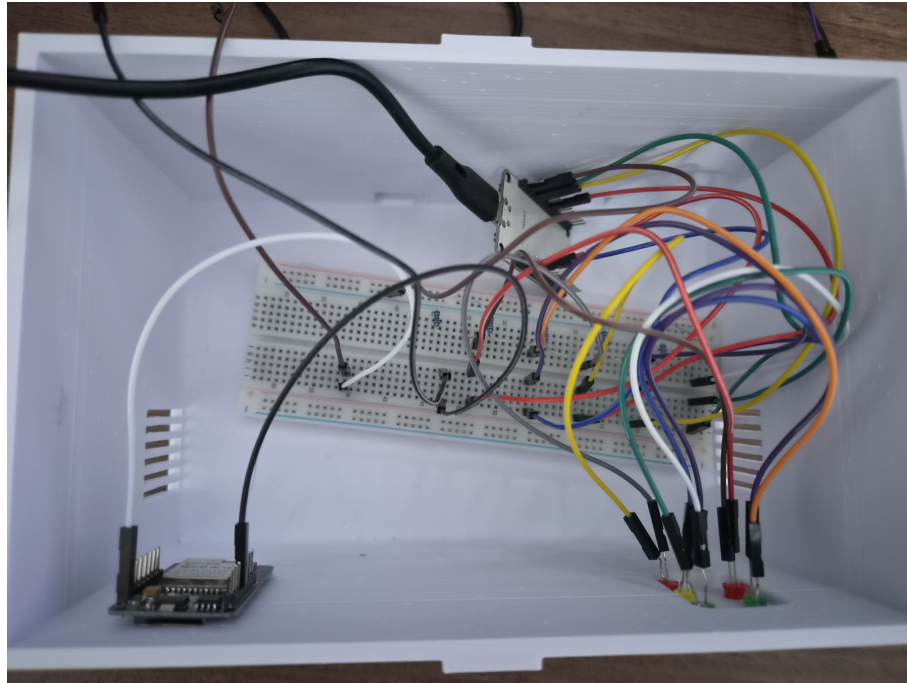


Figura 30 – Vista superior do compartimento que abriga os módulos de visão e o controlador.
Fonte: Autoria própria.

4.1 Módulo visual

A *ESP32-CAM* foi capaz de transmitir continuamente o fluxo de vídeo no formato *MJPEG* através de um servidor *HTTP* embarcado, mantendo conexão quase consistente com o computador responsável pelo processamento das imagens. A configuração adotada permitiu disponibilizar um fluxo contínuo de imagens suficiente para a aplicação proposta, no entanto, foram observadas limitações relacionadas à estabilidade e à taxa de atualização do fluxo em determinadas condições de rede. Esses resultados demonstram que dispositivos embarcados podem ser empregados como sensores visuais em sistemas de automação, desde que sejam consideradas as limitações de processamento, transmissão e qualidade da infraestrutura de comunicação. Vale também ressaltar que foi necessária uma alimentação contínua e estável de 5 V para garantir o funcionamento adequado do microprocessador durante a transmissão, pequenas oscilações na energia e níveis insuficientes podem aumentar, e aumentaram, o atraso na transmissão das imagens.

4.2 Módulo de processamento

A detecção de pedestres apresentou funcionamento consistente durante os testes realizados conforme Figura 31, onde o modelo *YOLOv8n* foi capaz de identificar corretamente a presença de pedestres nas imagens utilizadas para validação do protótipo, permitindo a geração da variável lógica utilizada pelo controlador semafórico.

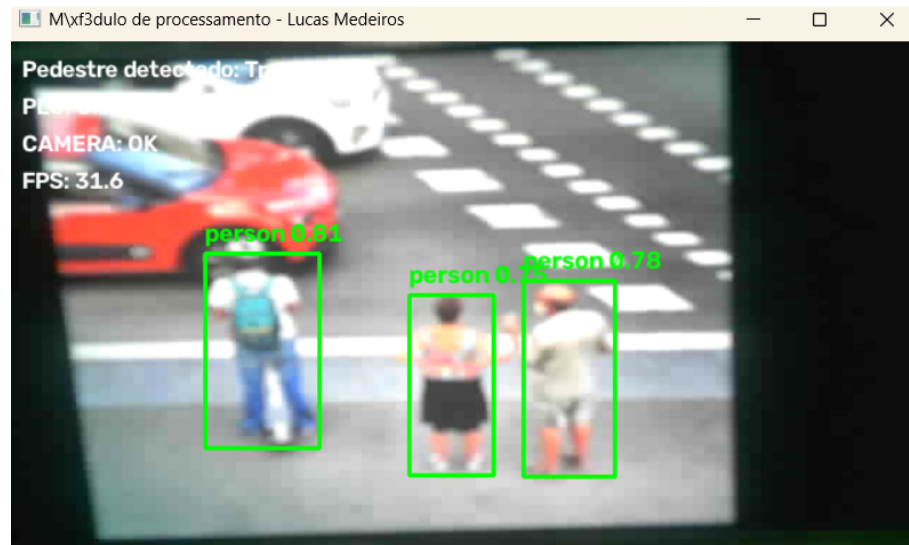


Figura 31 – Primeiro teste de detecção utilizando uma imagem fixa. Fonte: (Dreamstime, 2025).

Os principais resultados obtidos durante a execução do módulo de processamento são apresentados na Tabela 1. O tempo médio total de processamento foi de 22,29 ms por quadro, sendo que a inferência realizada pelo *YOLOv8n* representou a maior parcela desse intervalo. A taxa média de processamento observada foi de 31,2 *frames* por segundo, o que, durante os experimentos realizados, se mostrou como um valor suficiente para garantir uma boa margem de detecção em tempo real. Esses valores evidenciam que a etapa de inferência da rede neural constitui o principal fator limitante do processamento, enquanto as demais operações apresentaram contribuição significativamente menor.

Tabela 1 – Resumo das métricas de desempenho do sistema. Fonte: Autoria própria.

Métrica	Valor médio
Tempo de captura	0,30 ms
Tempo de pré-processamento	0,48 ms
Tempo de inferência (YOLOv8)	21,50 ms
Tempo de comunicação Modbus	0,01 ms
Tempo total de processamento	22,29 ms
FPS médio	31,2 FPS
Uso de CPU	62–79%
Uso de RAM	66,7%

A distribuição do tempo de processamento apresentada na Tabela 2 permite observar de maneira mais detalhada a contribuição de cada etapa. A inferência do *YOLOv8* correspondeu a aproximadamente 96,45% do tempo total, enquanto o pré-processamento representou 2,15% e a captura dos quadros, 1,35%. A comunicação *Modbus/TCP*, por sua vez, representou apenas 0,05% do tempo total medido. Dessa forma, os resultados indicam que eventuais melhorias de desempenho devem priorizar a etapa de inferência da

rede neural, uma vez que otimizações exclusivamente relacionadas à comunicação ou ao pré-processamento teriam impacto limitado sobre o tempo total de execução.

Tabela 2 – Participação percentual de cada etapa no tempo total de processamento. Fonte: Autoria própria.

Etapa	Tempo médio (ms)	Participação
Captura do quadro	0,30	1,35%
Pré-processamento	0,48	2,15%
Inferência YOLO	21,50	96,45%
Comunicação Modbus	0,01	0,05%
Total	22,29	100%

Além da detecção, o sistema incorporou mecanismos de supervisão destinados a aumentar a confiabilidade da comunicação entre os módulos. O mecanismo de *heartbeat* foi utilizado pelo controlador para verificar continuamente se o programa de processamento permanecia em execução, enquanto a variável *CameraOk* foi utilizada para indicar o estado da comunicação com a *ESP32-CAM*. Durante os testes realizados, ambas as funcionalidades apresentaram o comportamento esperado, permitindo que o controlador identificasse situações de perda de comunicação com o módulo de processamento ou com a câmera.

A utilização de recursos computacionais também foi considerada na avaliação do módulo. Conforme apresentado na Tabela 1, a utilização da CPU permaneceu entre 62% e 79%, enquanto a utilização de memória *RAM* permaneceu próxima de 66,7%. No maior nível de utilização observado, aproximadamente 21% da capacidade de processamento permaneceu disponível, enquanto cerca de 33% da memória *RAM* permaneceu livre. Essa margem foi suficiente para manter o sistema operacional, os processos de comunicação, a interface gráfica do *OpenCV* e as demais atividades executadas durante os testes sem ocorrência de falhas atribuídas à falta de recursos.

Entretanto, a margem disponível de processamento deve ser considerada na expansão do sistema. A inclusão de técnicas adicionais de processamento de imagens, especialmente aquelas destinadas à operação em condições de baixa luminosidade, ou a utilização de modelos de inteligência artificial mais complexos, aumentaria a demanda computacional. Isso foi observado durante os testes com técnicas de melhoria de imagem para visão noturna, que apresentaram custo computacional adicional e, por esse motivo, não foram incorporadas à versão final do protótipo. Assim, os resultados apresentados nas Tabelas 1 e 2 indicam que o principal caminho para a evolução do módulo está relacionado à otimização ou substituição da etapa de inferência.

4.3 Módulo de controle

No controlador desenvolvido em linguagem *Ladder* foi implementada toda a lógica responsável pelo ciclo semaforico e pelo tratamento das informações recebidas do sistema de visão computacional, que operaram conforme o previsto. Além do controle convencional dos tempos para veículos e pedestres, o mecanismo de supervisão (modo seguro) que utiliza temporizadores e detecção de variáveis *heartbeat* e *cameraOK* provenientes do módulo de processamento foi implementado com sucesso e se mostrou resistente contra situações adversas nos cenários de teste. Na situação de ausência de atualizações do módulo de processamento durante um intervalo predeterminado ou detecção de perda da comunicação com a câmera, é acionada automaticamente a condição de falha e a luz amarela permanece piscando até que a falha seja corrigida.

Por sua vez, os testes de desconexão e reconexão dos módulos de visão e processamento demonstraram que o controlador foi capaz de identificar as condições de falha e retornar à operação após o restabelecimento da comunicação, sem necessidade de reinicialização manual. Dessa forma, o microcontrolador cumpriu a função de representar, em escala experimental, a camada de decisão e atuação de um controlador semaforico, mantendo separadas as etapas de detecção e de controle.

4.4 Desafios

Embora o funcionamento geral tenha sido considerado adequado para a validação da proposta, alguns desafios foram identificados durante os testes experimentais. O principal deles está relacionado ao desempenho do sistema visual, que apresentou limitações na transmissão do fluxo de imagens utilizando a arquitetura de hardware e infraestrutura de rede disponíveis. Essas limitações foram observadas principalmente pela redução da taxa de atualização efetiva e pelo aumento da latência da comunicação entre o computador e a *ESP32-CAM*. Apesar de não impedirem a demonstração do conceito, transmissões mais estáveis e com menor latência poderiam proporcionar respostas mais rápidas e aumentar a capacidade do sistema em situações de maior fluxo de pessoas.

Observou-se ainda que a utilização de técnicas adicionais de melhoria de imagem para operação noturna, como correção gama, provocou redução ainda maior no desempenho computacional. Como o processamento já apresentava utilização significativa da *CPU*, conforme indicado na Tabela 1, a incorporação dessas técnicas poderia reduzir ainda mais a taxa de processamento. Por esse motivo, o modo de visão noturna não foi incorporado à versão final, permanecendo como possibilidade de evolução mediante a utilização de técnicas mais eficientes ou plataformas computacionais com maior capacidade.

Outro aspecto observado durante os experimentos refere-se ao próprio modelo de inteligência artificial utilizado para detecção de pedestres. Embora o modelo *YOLOv8n*

pré-treinado tenha apresentado resultados consistentes para a demonstração da proposta, verificou-se que sua precisão ainda pode ser aprimorada por meio do treinamento utilizando bases de dados específicas para ambientes urbanos semelhantes ao cenário considerado neste trabalho. A Figura 32 apresenta um exemplo dessa limitação, no qual ocorreu uma detecção incorreta de um hidrante como pessoa, mostrando que a utilização de um modelo genérico pode produzir falsos positivos em determinadas condições.

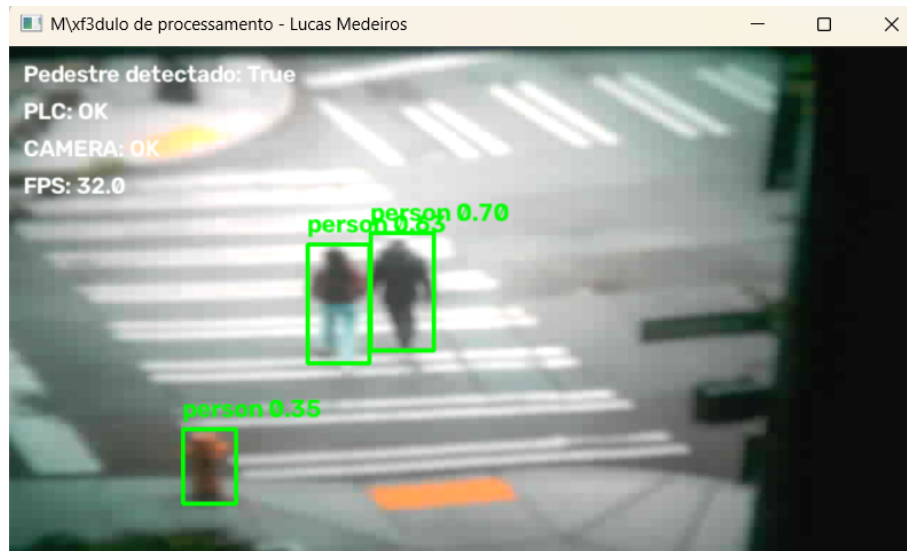


Figura 32 – Segundo teste de detecção utilizando uma imagem fixa. Fonte: (Josh Hild, 2025)

Nesse aspecto, a utilização de uma botoeira apresenta uma vantagem importante, já a solicitação gerada diretamente pela ação física do usuário demonstra que seu acionamento possui uma lógica extremamente simples e determinada. Não há necessidade de processamento de imagens, treinamento de modelos de inteligência artificial ou avaliação da qualidade da imagem. Porém, essa característica também representa uma limitação, pois a botoeira depende de que o pedestre perceba sua existência, consiga alcançá-la e efetivamente realize o acionamento. A visão computacional, quando corretamente configurada, permite substituir essa ação por uma detecção automática da presença, possibilitando que a demanda seja identificada sem interação física direta.

4.5 Custos de implementação

Sob o ponto de vista econômico, os resultados obtidos indicam que a proposta apresenta custo de implementação relativamente reduzido quando comparada aos sistemas comerciais de monitoramento semafórico. Considerando os componentes utilizados neste protótipo, o custo de uma unidade experimental é composto principalmente por uma *ESP32-CAM* (aproximadamente R\$ 60,00), uma placa *ESP32* para execução do OpenPLC (cerca de R\$ 50,00), módulo conversor de alimentação, *LEDs*, resistores, protoboard e

demais componentes eletrônicos, totalizando aproximadamente R\$ 200,00 em hardware, desconsiderando o computador utilizado para processamento, conforme buscas em sites como Mercado Livre e AliExpress, conforme Tabela 3 ([AliExpress, 2026](#); [Mercado Livre, 2026](#)).

Tabela 3 – Estimativa de custos do protótipo desenvolvido. Fonte: ([AliExpress, 2026](#); [Mercado Livre, 2026](#))

Componente	Valor aproximado (R\$)
ESP32-CAM	60
ESP32	50
Demais componentes (LEDs, resistores, protoboard, impressão 3d, etc.)	200

Em uma aplicação real, entretanto, seria recomendável substituir o computador convencional por equipamentos industriais ou plataformas dedicadas de computação de borda ou computadores industriais baseados em processadores *Intel Core*, cujos custos podem variar entre aproximadamente R\$ 2.500,00 e R\$ 8.000,00 por interseção, dependendo da capacidade de processamento requerida ([Advantech, 2026](#); [AAEON, 2026](#)). Da mesma forma, a *ESP32-CAM* poderia ser substituída por câmeras *IP* industriais ou câmeras de monitoramento em alta definição, com resoluções *Full HD* ou superiores e melhor desempenho em condições de baixa luminosidade. Equipamentos desse tipo normalmente apresentam valores entre R\$ 400,00 e R\$ 1.500,00 por unidade, oferecendo imagens de maior qualidade, maior confiabilidade operacional e melhor desempenho para algoritmos de visão computacional, conforme Tabela 4 ([Intelbras, 2026](#)).

Tabela 4 – Comparação entre o protótipo desenvolvido e uma implementação para aplicação real. Fonte: ([Advantech, 2026](#); [AAEON, 2026](#); [Intelbras, 2026](#)).

Item	Valor aproximado (R\$)
Computadores industriais	2.500,00 a 8.000,00
Câmeras IP industriais	400,00 a 1.500,00
Controladores industriais	Próprio semáforo

Para avaliar melhor a proposta, é importante comparar esse valor não apenas com sistemas de visão computacional de maior porte, mas também com uma solução convencional, como a botoeira de pedestres. Processos de aquisição pública demonstram que o preço de uma botoeira pode variar significativamente conforme suas características, fornecimento e instalação. Em uma aquisição pública realizada no ano de 2023 no Estado de Minas Gerais, por exemplo, foi registrado valor estimado para o item de aproximadamente R\$ 300,00 a R\$ 312,00 ([GOVERNO DO ESTADO DE MINAS GERAIS, 2023](#)). Em outro processo de aquisição pública, uma botoeira convencional completa apresentou valor de R\$ 414,00 por unidade ([Prefeitura de Joinville, 2026](#)). Esses valores demonstram que a

botoeira individualmente apresenta custo inferior ao conjunto necessário para implementar a solução de visão computacional.

4.6 Considerações finais

Os resultados apresentados ao longo deste capítulo demonstram que a arquitetura desenvolvida foi capaz de integrar aquisição de imagens, processamento por inteligência artificial, comunicação em rede e controle lógico em um único sistema experimental. As métricas apresentadas nas Tabelas 1 e 2 permitiram quantificar o comportamento do módulo de processamento e identificar a inferência do *YOLOv8n* como a principal parcela do tempo de execução. Ao mesmo tempo, os testes realizados com *Heartbeat* e *CameraOk* demonstraram o funcionamento dos mecanismos de supervisão e tratamento de falhas implementados no controlador. A botoeira apresenta como principais vantagens o baixo custo, a simplicidade e a fácil integração aos controladores semafóricos, porém depende da ação física do pedestre. O sistema proposto elimina essa necessidade ao realizar a detecção automaticamente por visão computacional, o que pode ser especialmente útil para pessoas com limitações de mobilidade, usuários que tenham dificuldade de alcançar ou acionar o dispositivo, além de situações em que o pedestre não perceba a necessidade de acionamento ou evite tocar na botoeira por questões sanitárias. Dessa forma, a detecção automática permite que a presença do pedestre seja considerada pelo sistema sem depender de sua interação direta.

Embora a implementação do sistema proposto apresente inicialmente um custo superior ao de uma botoeira, principalmente devido à câmera, processamento e controlador utilizados, como evidenciado pela Tabela 4, essa diferença está relacionada à maior complexidade e ao caráter experimental do protótipo. Futuras melhorias, como modelos de inteligência artificial mais leves, otimização do processamento e utilização de equipamentos de menor custo e maior eficiência, podem reduzir o investimento necessário. Além disso, enquanto a botoeira possui essencialmente a função de solicitar a travessia, a infraestrutura de visão computacional pode futuramente ser utilizada para outras aplicações, como monitoramento de pedestres, análise do fluxo e identificação de diferentes situações no ambiente urbano, agregando funcionalidades que não seriam possíveis apenas com o acionamento convencional. Dessa forma, deve-se também considerar o potencial de desenvolvimento do projeto.

Além do mais, considerando o contexto onde diversos cruzamentos ainda operam com tempos fixos de abertura sem considerar a demanda real de veículos e pedestres, a adoção de sistemas inteligentes como o proposto pode contribuir para maior eficiência operacional, redução do tempo de espera e melhor aproveitamento dos ciclos semafóricos. Embora a implantação em larga escala exija equipamentos mais robustos e adequações aos padrões utilizados pelos órgãos responsáveis pela sinalização viária, os resultados obtidos

demonstram que a arquitetura proposta é tecnicamente viável e pode servir como base para futuras soluções de controle semaforico inteligente.

De maneira geral, os resultados alcançados atenderam aos objetivos estabelecidos para este trabalho. Foi possível demonstrar, em um ambiente de laboratório, a integração entre técnicas de visão computacional, comunicação em rede, inteligência artificial e controladores programáveis, formando um sistema funcional capaz de detectar pedestres, processar as informações em tempo real e atuar automaticamente sobre um controlador semaforico. Apesar das limitações identificadas quanto ao desempenho e à necessidade de aperfeiçoamento do modelo de detecção, o protótipo confirmou a viabilidade prática da proposta e evidenciou o potencial dessa arquitetura para futuras aplicações em sistemas inteligentes de mobilidade urbana.

5 CONCLUSÃO

Este trabalho teve como objetivo desenvolver um protótipo de sistema semafórico inteligente baseado em visão computacional, integrando técnicas de processamento de imagens, inteligência artificial e controladores lógicos programáveis em um único sistema funcional. Ao longo dele, foi possível estruturar uma arquitetura capaz de reunir aquisição de imagens, processamento de informações e controle lógico em uma única solução, demonstrando a possibilidade de aplicação desses recursos em sistemas voltados à mobilidade urbana.

Os experimentos realizados permitiram verificar o funcionamento integrado dos diferentes elementos que compõem a proposta, bem como a comunicação entre eles. O sistema desenvolvido apresentou comportamento compatível com a finalidade estabelecida para o protótipo, permitindo demonstrar a utilização de informações obtidas por visão computacional para auxiliar na tomada de decisões de um sistema de controle semafórico.

Dessa forma, o trabalho permitiu demonstrar a aplicação conjunta de tecnologias de visão computacional, inteligência artificial, comunicação em rede e controle programável em uma proposta de automação voltada à mobilidade urbana. Os resultados obtidos fornecem uma base para a continuidade do desenvolvimento e para estudos futuros relacionados à utilização de sistemas inteligentes em aplicações de controle de tráfego.

REFERÊNCIAS

- AAEON. *Industrial Embedded Computers*. 2026. Disponível em: <<https://www.aaeon.com>>. Citado na página 39.
- Advantech. *Industrial Computers*. 2026. Disponível em: <<https://www.advantech.com>>. Citado na página 39.
- AliExpress. *ESP32 DevKit*. 2026. Pesquisa de preços realizada em julho de 2026. Disponível em: <<https://pt.aliexpress.com/w/wholesale-esp32-dev-kit.html>>. Citado na página 39.
- Autonomylogic. *OpenPLC Documentation*. 2024. <<https://autonomylogic.com>>. Acesso em: 21 nov. 2025. Citado 4 vezes nas páginas 8, 9, 14 e 22.
- BOLTON, W. *Programmable Logic Controllers*. 6. ed. Oxford: Elsevier, 2015. Citado 2 vezes nas páginas 8 e 10.
- Brasil. Departamento Nacional de Trânsito. *Manual Brasileiro de Sinalização de Trânsito: Volume V – Sinalização Semafórica*. Brasília: DENATRAN, 2014. Citado 4 vezes nas páginas 4, 6, 11 e 12.
- CET – Companhia de Engenharia de Tráfego. *Sistema de Controle Semafórico: Especificações Técnicas*. São Paulo, 2014. Disponível em: <https://www.cetsp.com.br/media/304246/especific_func_%20tec_crtl_tempofixo_rev3_preliminar.pdf>. Citado 2 vezes nas páginas 4 e 11.
- CET – Companhia de Engenharia de Tráfego. *Manuais de Sinalização Urbana*. São Paulo: CET, 2026. Disponível em: <<https://www.cetsp.com.br/consultas/publicacoes/manuais-de-sinalizacao-urbana.aspx>>. Citado 3 vezes nas páginas 4, 6 e 11.
- CIM automação. *CLP: O que é, como funciona e importância na automação*. 2017. Acesso em: 15 abril 2025. Disponível em: <<https://blog.cimautomacao.com.br/clp-o-que-e-como-funciona-e-importancia-na-automacao/>>. Citado na página 10.
- CONTRAN. *Sinalização Semafórica*. 2022. Disponível em: <https://www.gov.br/transportes/pt-br/assuntos/transito/arquivos-senatran/docs/copy_of____05____MBST_Vol._V____Sinalizacao_Semaforica.pdf>. Citado na página 12.
- Dreamstime. *Pessoas à espera para atravessar a rua. Os carros param em frente à passadeira*. 2025. Acesso em: 18 jul. 2026. Disponível em: <<https://pt.dreamstime.com/pessoas-%C3%A0-espera-de-atravesar-rua-os-carros-param-em-frente-travessia-zebra-image203276074>>. Citado na página 35.
- EMTU – Empresa Metropolitana de Transportes Urbanos. *Relatório de Mobilidade da Região Metropolitana de São Paulo*. 2022. <<https://emtu.sp.gov.br>>. Acesso em: 21 nov. 2025. Citado na página 1.

- Espressif Systems. *ESP32 Series Datasheet e ESP32-CAM Development Board*. 2024. <<https://www.espressif.com>>. Acesso em: 21 nov. 2025. Citado 5 vezes nas páginas 7, 14, 17, 47 e 48.
- GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*. 4. ed. New York: Pearson, 2018. Citado 3 vezes nas páginas 6, 7 e 10.
- GOVERNO DO ESTADO DE MINAS GERAIS. *Ata de Realização do Pregão Eletrônico N° 00106/2021 (SRP)*. 2023. Disponível em: <https://www.santaluzia.mg.gov.br/wp-content/uploads/2021/10/ATA-DE-REALIZACAO-DO-PREGAO-ELETRONICO-2.pdf?utm_source=>>. Citado na página 39.
- HARRISON, C.; ABBOTT, I. The role of smart cities in urban management. In: *Smart City Applications*. [S.l.]: Springer, 2015. Citado 3 vezes nas páginas 1, 3 e 28.
- IETF. *Hypertext Transfer Protocol (HTTP/1.1)*. 2022. Utilizado como base para transmissão contínua de imagens MJPEG. Acesso em: 18 jul. 2026. Disponível em: <<https://datatracker.ietf.org/doc/html/rfc9110>>. Citado na página 53.
- Intelbras. *Câmeras IP para videomonitoramento*. 2026. Disponível em: <<https://www.intelbras.com>>. Citado 3 vezes nas páginas 7, 17 e 39.
- International Electrotechnical Commission. *IEC 61131-3: Programmable controllers – Part 3: Programming languages*. 3. ed. Geneva: IEC, 2013. Citado na página 8.
- Josh Hild. *Duas pessoas atravessam uma rua em uma faixa de pedestres*. 2025. Acesso em: 18 jul. 2026. Disponível em: <<https://unsplash.com/pt-br/fotografias/duas-pessoas-atravessam-uma-rua-em-uma-faixa-de-pedestres-Llx4vSMw4zw>>. Citado na página 38.
- Mercado Livre. *ESP32-CAM AI Thinker*. 2026. Pesquisa de preços realizada em julho de 2026. Disponível em: <<https://lista.mercadolivre.com.br/esp32-cam>>. Citado na página 39.
- Modbus Organization. *MODBUS Application Protocol Specification V1.1b3*. 2012. Acesso em: 18 jul. 2026. Disponível em: <<https://modbus.org/specs.php>>. Citado 2 vezes nas páginas 9 e 52.
- NTCIP. *NTCIP 1202: Object Definitions for Actuated Traffic Signal Controller (ASC) Units*. Washington, DC, 2019. Citado 7 vezes nas páginas 5, 8, 9, 10, 12, 13 e 27.
- OpenCV Team. *OpenCV Documentation*. 2025. Acesso em: 18 jul. 2026. Disponível em: <<https://docs.opencv.org/4.x/>>. Citado na página 54.
- OpenPLC Project. *OpenPLC Programming Reference*. 2018. <<https://openplcproject.gitlab.io>>. Acesso em: 21 nov. 2025. Citado 2 vezes nas páginas 8 e 9.
- Organisation for Economic Co-operation and Development. *Urban Mobility Outlook*. 2019. <<https://oecd.org>>. Acesso em: 21 nov. 2025. Citado na página 1.
- PRABHU, A. *ESP32-CAM Pinout Reference*. 2026. <<https://lastminuteengineers.com/esp32-cam-pinout-reference/>>. Acesso em: 15 abril 2025. Citado na página 18.

PRABHU, A. *ESP32 Pinout Reference*. 2026. <<https://lastminuteengineers.com/esp32-pinout-reference/>>. Acesso em: 15 abril 2025. Citado na página 23.

Prefeitura de Joaçaba. *Pedestres terão que acionar botoeira do semáforo para liberar travessia em três pontos de Joaçaba*. 2022. Disponível em: <<https://joacaba.sc.gov.br/noticia-736106/>>. Citado na página 12.

Prefeitura de Joinville. *ERRATA SEI Nº 28362356/2026 - SAP.LCT*. 2026. Disponível em: <https://www.joinville.sc.gov.br/public/edital/anexo/3915b6e88c6022e01178efdf346dc9b0.pdf?utm_source=>>. Citado na página 39.

PyModbus Developers. *PyModbus Documentation*. 2025. Acesso em: 18 jul. 2026. Disponível em: <<https://pymodbus.readthedocs.io/>>. Citado 3 vezes nas páginas 52, 53 e 54.

QURISHEE, M. A. *What's inside a Traffic Signal Controller? Part-1*. 2020. <https://www.youtube.com/watch?v=BydjVA_Q-1c>. Acesso em: 15 abril 2025. Citado na página 5.

REDMON, J. *YOLO: Real-Time Object Detection*. 2018. <<https://pjreddie.com/darknet/yolo/>>. Acesso em: 15 abril 2025. Citado na página 6.

Requests Developers. *Requests: HTTP for Humans*. 2025. Acesso em: 18 jul. 2026. Disponível em: <<https://requests.readthedocs.io/>>. Citado na página 53.

SZELISKI, R. *Computer Vision: Algorithms and Applications*. 2. ed. [S.l.]: Springer, 2022. Citado 3 vezes nas páginas 14, 20 e 21.

Ultralytics. *Ultralytics YOLOv8 Docs*. 2024. <<https://docs.ultralytics.com>>. Acesso em: 21 nov. 2025. Citado 5 vezes nas páginas 6, 7, 14, 21 e 52.

VASCONCELLOS, E. A. *Mobilidade Urbana: Desafios e Alternativas*. São Paulo: Annablume, 2014. Citado na página 1.

World Bank. *The Cost of Traffic Congestion in Urban Areas*. 2020. <<https://worldbank.org>>. Acesso em: 21 nov. 2025. Citado na página 1.

XIMENES, T. S. dos S. et al. Vehicular traffic flow detection and monitoring for implementation of smart traffic light: A case study for road intersection in limeira, brazil. *Future Transportation*, v. 4, n. 4, p. 1388–1401, 2024. ISSN 2673-7590. Disponível em: <<https://www.mdpi.com/2673-7590/4/4/67>>. Acesso em: 25 nov. 2025. Citado 2 vezes nas páginas 2 e 12.

Apêndices

APÊNDICE A – CÓDIGO DA ESP32-CAM

O código [A.1](#) desenvolvido para a *ESP32-CAM* foi baseado diretamente no exemplo *CameraWebServer*, disponibilizado pela *Espressif* como parte do pacote de exemplos da plataforma *ESP32*. Esse exemplo fornece toda a infraestrutura necessária para inicialização da câmera, configuração do servidor *HTTP* e transmissão de imagens pela rede. Dessa forma, optou-se por utilizá-lo como base devido à sua estabilidade, ampla documentação e compatibilidade com a placa empregada neste projeto. As modificações tiveram como objetivo simplificar o funcionamento do sistema e adaptá-lo às necessidades específicas do protótipo, eliminando funcionalidades desnecessárias e priorizando o desempenho na transmissão de imagens ([Espressif Systems, 2024](#)).

Uma das alterações realizadas em relação ao programa original foi a incorporação das definições de hardware diretamente no arquivo principal. No exemplo disponibilizado pela *Espressif*, os pinos referentes ao *ESP32-CAM* encontram-se separados no arquivo *board_config.h*, permitindo que um mesmo programa seja utilizado com diferentes modelos de câmeras. Como neste projeto foi utilizada apenas o modelo *AI-Thinker*, optou-se por transferir essas definições para o próprio código-fonte, tornando o programa autocontido e facilitando sua compreensão, manutenção e reprodução durante o desenvolvimento do trabalho.

Outra modificação consistiu na simplificação do servidor web. O exemplo original disponibiliza uma interface completa acessível pelo navegador, contendo páginas *HTML*, controles para ajuste de brilho, contraste, resolução, qualidade da imagem, rotação, espelhamento e diversas outras configurações do sensor, além da visualização do vídeo em tempo real. Entretanto, como o objetivo deste trabalho não envolve interação manual com a câmera, mas apenas o fornecimento contínuo de imagens para o módulo de processamento, toda essa interface foi removida. Em seu lugar, foi implementado apenas o serviço responsável pela transmissão do fluxo *MJPEG* através do endereço *stream*, reduzindo a quantidade de processamento executada pelo microcontrolador.

Também foram realizadas adaptações na configuração do sensor de imagem, enquanto o exemplo original inicializa a câmera utilizando resolução elevada (*UXGA*), qualidade *JPEG* variável e ajustes automáticos em função da disponibilidade de memória, neste projeto foram adotados parâmetros fixos que foram definidos para obter um equilíbrio entre qualidade da imagem, estabilidade da transmissão e desempenho do sistema. A resolução foi configurada para *VGA*, o formato *JPEG* foi mantido por apresentar menor consumo de largura de banda e os parâmetros de brilho, contraste e saturação permaneceram em valores neutros, permitindo que eventuais tratamentos de imagem fossem realizados posteriormente no módulo de processamento, onde há maior

capacidade computacional ([Espressif Systems, 2024](#)).

Por fim tivemos a remoção de funcionalidades específicas para aplicações de demonstração, como configuração automática de *LED* de iluminação, ajustes específicos para determinados sensores e modelos de placas, além de otimizações voltadas para reconhecimento facial. Como o protótipo desenvolvido realiza exclusivamente a aquisição e transmissão das imagens, esses recursos deixaram de ser necessários e sua remoção contribuiu para tornar o programa mais enxuto.

```

1 #include <WiFi.h>
2 #include "esp_camera.h"
3 #include "esp_http_server.h"
4
5 //CONFIGURACOES DE REDE
6 const char* ssid = "";
7 const char* password = "";
8
9 //PINOS PARA AI-THINKER
10 #define PWDN_GPIO_NUM    32
11 #define RESET_GPIO_NUM   -1
12 #define XCLK_GPIO_NUM     0
13 #define SIOD_GPIO_NUM    26
14 #define SIOC_GPIO_NUM    27
15 #define Y9_GPIO_NUM      35
16 #define Y8_GPIO_NUM      34
17 #define Y7_GPIO_NUM      39
18 #define Y6_GPIO_NUM      36
19 #define Y5_GPIO_NUM      21
20 #define Y4_GPIO_NUM      19
21 #define Y3_GPIO_NUM      18
22 #define Y2_GPIO_NUM       5
23 #define VSYNC_GPIO_NUM   25
24 #define HREF_GPIO_NUM    23
25 #define PCLK_GPIO_NUM    22
26
27 httpd_handle_t stream_httpd = NULL;
28
29 // stream MJPEG
30 static esp_err_t stream_handler(httpd_req_t *req) {
31     camera_fb_t *fb = NULL;
32     esp_err_t res = ESP_OK;
33     uint8_t *_jpg_buf = NULL;
34     size_t _jpg_buf_len = 0;
35     char part_buf[128];
36     static const char* STREAM_BOUNDARY = "123456789000000000000987654321";
37     static const char* STREAM_CONTENT_TYPE =
38         "multipart/x-mixed-replace;boundary=123456789000000000000987654321";
39     static const char* STREAM_BOUNDARY_STR =

```

```
    "\r\n--123456789000000000000987654321\r\n";
39 static const char* STREAM_PART = "Content-Type:
    image/jpeg\r\nContent-Length: %u\r\n\r\n";
40
41 httpd_resp_set_type(req, STREAM_CONTENT_TYPE);
42 httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
43
44 while (true) {
45     fb = esp_camera_fb_get();
46     if (!fb) {
47         Serial.println("Falha na captura");
48         res = ESP_FAIL;
49         break;
50     }
51
52     _jpg_buf_len = fb->len;
53     _jpg_buf = fb->buf;
54
55     // Envia as partes do stream
56     if (httpd_resp_send_chunk(req, STREAM_BOUNDARY_STR,
        strlen(STREAM_BOUNDARY_STR)) != ESP_OK) break;
57     snprintf(part_buf, sizeof(part_buf), STREAM_PART, _jpg_buf_len);
58     if (httpd_resp_send_chunk(req, part_buf, strlen(part_buf)) !=
        ESP_OK) break;
59     if (httpd_resp_send_chunk(req, (const char*)_jpg_buf, _jpg_buf_len)
        != ESP_OK) break;
60
61     esp_camera_fb_return(fb);
62     fb = NULL;
63     _jpg_buf = NULL;
64 }
65
66 if (fb) esp_camera_fb_return(fb);
67 return res;
68 }
69
70 void startCameraServer() {
71     httpd_config_t config = HTTPD_DEFAULT_CONFIG();
72     config.server_port = 80;
73     config.max_uri_handlers = 4;
74     config.lru_purge_enable = true; // Para liberar conexoes antigas
        automaticamente
75
76     httpd_uri_t stream_uri = {
77         .uri = "/stream",
78         .method = HTTP_GET,
79         .handler = stream_handler,
```

```
80     .user_ctx = NULL
81 };
82
83 if (httpd_start(&stream_httpd, &config) == ESP_OK) {
84     httpd_register_uri_handler(stream_httpd, &stream_uri);
85     Serial.print("Stream disponivel em: http://");
86     Serial.print(WiFi.localIP());
87     Serial.println("/stream");
88 } else {
89     Serial.println("Falha ao iniciar o servidor");
90 }
91 }
92
93 void setup() {
94     Serial.begin(115200);
95     delay(1000);
96     Serial.println("\n\nIniciando camera...");
97
98     // Configuracao da camera
99     camera_config_t config;
100     config.ledc_channel = LEDC_CHANNEL_0;
101     config.ledc_timer = LEDC_TIMER_0;
102     config.pin_d0 = Y2_GPIO_NUM;
103     config.pin_d1 = Y3_GPIO_NUM;
104     config.pin_d2 = Y4_GPIO_NUM;
105     config.pin_d3 = Y5_GPIO_NUM;
106     config.pin_d4 = Y6_GPIO_NUM;
107     config.pin_d5 = Y7_GPIO_NUM;
108     config.pin_d6 = Y8_GPIO_NUM;
109     config.pin_d7 = Y9_GPIO_NUM;
110     config.pin_xclk = XCLK_GPIO_NUM;
111     config.pin_pclk = PCLK_GPIO_NUM;
112     config.pin_vsync = VSYNC_GPIO_NUM;
113     config.pin_href = HREF_GPIO_NUM;
114     config.pin_sscb_sda = SIOD_GPIO_NUM;
115     config.pin_sscb_scl = SIOC_GPIO_NUM;
116     config.pin_pwdn = PWDN_GPIO_NUM;
117     config.pin_reset = RESET_GPIO_NUM;
118     config.xclk_freq_hz = 20000000;
119     config.pixel_format = PIXFORMAT_JPEG;
120     config.frame_size = FRAMESIZE_VGA;
121     config.jpeg_quality = 25;
122     config.fb_count = 1;
123     config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
124     config.fb_location = CAMERA_FB_IN_PSRAM;
125
126     esp_err_t err = esp_camera_init(&config);
```

```
127  if (err != ESP_OK) {
128      Serial.printf("Falha na camera: erro 0x%x\n", err);
129      while (1) delay(1000);
130  }
131  Serial.println("Camera OK");
132
133  sensor_t *s = esp_camera_sensor_get();
134  s->set_quality(s, 25);
135  s->set_brightness(s, 0);
136  s->set_contrast(s, 0);
137  s->set_saturation(s, 0);
138
139  WiFi.begin(ssid, password);
140  Serial.print("Conectando WiFi");
141  while (WiFi.status() != WL_CONNECTED) {
142      delay(500);
143      Serial.print(".");
144  }
145  Serial.println("\nWiFi conectado!");
146  Serial.print("IP: ");
147  Serial.println(WiFi.localIP());
148
149  startCameraServer();
150 }
151
152 void loop() {
153     delay(1000);
154 }
```

Excerto de código A.1 – Código de configuração da ESP32-CAM para transmissão de imagens.

APÊNDICE B – CÓDIGO DO MÓDULO DE PROCESSAMENTO

A seguir, apresenta-se a descrição do Código B.1, responsável pela implementação do bloco de processamento do sistema proposto. Esse programa atua como elemento intermediário entre o módulo visual e o controlador semafórico, sendo encarregado de receber o fluxo de vídeo transmitido pela *ESP32-CAM*, analisar os quadros obtidos, identificar a presença de pedestres e converter o resultado em uma variável lógica enviada ao controlador *OpenPLC* por meio do protocolo *Modbus/TCP*.

Inicialmente, é feita a importação das bibliotecas necessárias à execução do programa. As bibliotecas *time* e *threading* são utilizadas para controle temporal e execução concorrente de tarefas, permitindo que a leitura do fluxo de vídeo e a comunicação com o controlador ocorram simultaneamente ao processamento das imagens. A biblioteca *requests* é responsável por acessar o stream *MJPEG* disponibilizado pela *ESP32-CAM* através de requisições *HTTP* contínuas. As bibliotecas *cv2* (*OpenCV*) e *numpy* são utilizadas para decodificação e manipulação das imagens recebidas. A biblioteca *Ultralytics* fornece a implementação do modelo de detecção *YOLOv8* utilizado no projeto, enquanto *pymodbus* permite a comunicação *Modbus/TCP* com o controlador semafórico. As bibliotecas *psutil* e *collections.deque* são empregadas para monitoramento de recursos do sistema e medição de desempenho, respectivamente.

Na sequência, são definidos os parâmetros gerais de configuração do sistema, como o endereço *IP* do controlador *OpenPLC*, as bobinas *Modbus* associadas ao sinal de presença de pedestres, *heartbeat* e *status* da câmera, a *URL* do fluxo de vídeo da *ESP32-CAM* e o caminho do modelo *YOLO* utilizado na detecção. Também são definidos parâmetros de pré-processamento das imagens, como resolução final, ajustes de brilho e contraste e inversão vertical da imagem. Além disso, são estabelecidos valores utilizados na lógica de histerese temporal, que determina quantos quadros consecutivos devem confirmar a presença ou ausência de pedestres antes que o estado lógico seja alterado, bem como o intervalo de escrita no *PLC* e o período de verificação da conexão ([Ultralytics, 2024](#); [PyModbus Developers, 2025](#)).

A função *conecta_plc()* é responsável por estabelecer a conexão entre o computador que executa o processamento e o controlador *OpenPLC*. Ela cria um cliente *Modbus/TCP* utilizando o endereço *IP* e a porta do controlador, realiza a tentativa de conexão com o *PLC* e retorna o objeto cliente para que outras partes do programa possam enviar comandos ao controlador ([PyModbus Developers, 2025](#); [Modbus Organization, 2012](#)). Já a função *escreve_bobina()* realiza a escrita da variável lógica no controlador de maneira

segura, tratando possíveis falhas de comunicação. Ela envia ao *PLC* o valor lógico correspondente à presença ou ausência de pedestres e verifica se a operação foi concluída corretamente no protocolo *Modbus*. Em caso de falha, ela registra uma mensagem de erro e impede que o programa interrompa sua execução. No código existem três variáveis, *COIL_PEDESTRIAN*, *COIL_HEARTBEAT* e *COIL_CAMERA_STATUS*, que representam as fontes de recebimento de dados no controlador, sendo elas responsáveis, respectivamente, pelos dados da presença de pedestres, dados de heartbeat e dados de status da câmera. O controlador (*ESP32*) é programado através do *OpenPLC* para receber esses dados ([PyModbus Developers, 2025](#)).

Para reduzir a latência no recebimento do vídeo, foi criada a classe *LeitorStream*, responsável pela leitura contínua e otimizada do fluxo *MJPEG* transmitido pela câmera. Essa classe utiliza uma linha de execução dedicada para receber os dados do stream e reconstruir as imagens *JPEG* a partir dos bytes recebidos via rede. Ela opera estabelecendo uma conexão *HTTP* contínua com a *URL* do stream, onde recebe blocos de dados binários contendo partes das imagens *JPEG*, identifica os marcadores de início e fim de cada imagem dentro do fluxo, reconstrói os quadros *JPEG* e os decodifica em imagens utilizáveis pelo *OpenCV*. O quadro mais recente é armazenado para uso pelo laço principal do programa, enquanto a inversão vertical é aplicada diretamente no momento da captura para reduzir processamento posterior ([Requests Developers, 2025](#); [IETF, 2022](#)). Adicionalmente, a classe monitora a taxa de quadros recebidos da stream para fins de diagnóstico, armazenando as medições em uma estrutura de dados do tipo *deque* (estrutura de dados linear que permite inserir e remover elementos em qualquer uma de suas extremidades) para cálculo da média móvel.

Para garantir a comunicação estável e sem bloqueios com o controlador, foi implementada a classe *EscritorModbus*, que opera em uma linha de execução separada dedicada exclusivamente às operações de escrita *Modbus*. Essa classe mantém uma fila interna onde os dados a serem transmitidos são enfileirados pelo laço principal, permitindo que o processamento de imagens prossiga sem interrupções causadas por latências de rede ou indisponibilidade temporária do *PLC*. A *thread* monitora continuamente a conexão com o controlador, realizando reconexões automáticas em caso de falha, e processa os itens da fila em intervalos regulares definidos pelo parâmetro *PLC_WRITE_PERIOD*. Caso a fila atinja sua capacidade máxima, os dados mais recentes são descartados para evitar o bloqueio do programa, priorizando a fluidez do processamento visual em detrimento de eventuais perdas de comunicação. Isso reduz significativamente o tempo de processamento por quadro e aumenta a resistência do sistema a falhas de comunicação ([PyModbus Developers, 2025](#)).

A função *prepara_frame()* executa o pré-processamento da imagem antes da etapa de inferência do modelo de visão computacional. Ela redimensiona a imagem para resolução padronizada e ajusta o brilho e o contraste (linhas 46 e 45, respectivamente) para melhorar

a qualidade visual do quadro analisado, compensando as condições de iluminação do ambiente (OpenCV Team, 2025).

A função principal do programa, chamada *main()*, coordena todo o fluxo de execução do sistema, onde inicialmente o modelo *YOLO* é carregado, o leitor do stream *MJPEG* é iniciado e a thread de escrita *Modbus* é ativada. Em seguida, o programa inicializa outras variáveis utilizadas no controle de estado e no cálculo da taxa de quadros processados, além de criar o arquivo para registro das métricas de desempenho. Dentro do laço principal, o sistema obtém repetidamente o quadro mais recente disponível do stream e aplica o pré-processamento. Esse quadro é então enviado ao modelo *YOLO*, configurado para detectar apenas a classe correspondente a pessoas. Caso o modelo identifique ao menos uma pessoa na imagem, o sistema considera que há pedestres na cena.

Para evitar oscilações rápidas entre estados de presença e ausência de pedestres, o programa aplica um mecanismo de histerese. Nesse mecanismo, a mudança de estado só ocorre após um número mínimo de quadros consecutivos (3 para quando há confirmação de presença e 6 para quando não há), confirmando a condição observada, aumentando a estabilidade do sistema e reduz efeitos causados por falsas detecções. Após a determinação do estado lógico correspondente à presença de pedestres, o programa enfileira os dados para envio ao controlador através da linha de execução da *Modbus*, que se encarrega de transmiti-los no intervalo adequado. O programa também calcula continuamente a taxa de quadros processados por segundo, coleta métricas de uso de *CPU* e memória *RAM*, e exibe na tela informações de diagnóstico, como estado da detecção de pedestres, estado da comunicação com o *PLC*, estado da conexão com o stream de vídeo e tamanho atual da fila de comunicação. As métricas de desempenho são registradas periodicamente em um arquivo de texto para análise posterior, permitindo avaliar a eficiência do sistema ao longo do tempo (PyModbus Developers, 2025).

```
1 import time
2 import threading
3 import requests
4 import cv2
5 import numpy as np
6 from ultralytics import YOLO
7 from pymodbus.client import ModbusTcpClient
8 import psutil
9 import os
10 from datetime import datetime
11 import queue
12 from collections import deque
13
14 # ----- CONFIGURACOES -----
15
16 PLC_IP = "10.236.29.136"
17 PLC_PORT = 502
```

```
18
19
20 COIL_PEDESTRIAN = 5
21 COIL_HEARTBEAT = 6
22 COIL_CAMERA_STATUS = 7
23
24 STREAM_URL = "http://10.236.29.32/stream"
25
26 MODEL_PATH = "yolov8n.pt"
27 CONF_THRES = 0.30
28 YOLO_IMGSZ = 320
29 YOLO_DEVICE = "cpu"
30
31 # classe que o yolo detecta como pessoa
32 COCO_PERSON = 0
33
34 ON_FRAMES_REQUIRED = 3
35 OFF_FRAMES_REQUIRED = 6
36
37 PLC_WRITE_PERIOD = 0.20
38 PLC_WRITE_INTERVAL = 3.0
39
40 #tratamento
41 FRAME_W = 640
42 FRAME_H = 360
43 APPLY_VERTICAL_FLIP = True
44 APPLY_CONTRAST_BRIGHTNESS = True
45 ALPHA_CONTRAST = 1.08
46 BETA_BRIGHTNESS = 6
47
48 HTTP_TIMEOUT = (3, 5)
49 JPEG_START = b"\xff\xd8"
50 JPEG_END = b"\xff\xd9"
51
52 #metricas
53 METRICS_WRITE_INTERVAL = 5
54 metrics_file =
55     f"metricas_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt"
56
57 modbus_queue = queue.Queue(maxsize=10)
58
59 estado = {
60     'pedestre': False,      # se tem pedestre detectado
61     'heartbeat': False,    # heartbeat
62     'camera_ok': False,    # se a camera ta online
63     'plc_ok': False,       # se o PLC ta respondendo
64     'ultima_escrita': 0,   # quando escreveu no PLC pela ultima vez
```

```
64     'contador_on': 0,          # quantos frames seguidos com pedestre
65     'contador_off': 0,        # quantos frames seguidos sem pedestre
66     'pessoa_detectada': False # deteccao do yolo
67 }
68
69 # ----- PLC -----
70
71 def conecta_plc():
72     cliente = ModbusTcpClient(PLC_IP, port=PLC_PORT, timeout=1.0)
73     cliente.connect()
74     return cliente
75
76 def escreve_bobina(cliente, endereco, valor):
77     try:
78         resposta = cliente.write_coil(endereco, valor)
79         return not resposta.isError()
80     except Exception as e:
81         print(f"erro ao escrever bobina {endereco}: {e}")
82         return False
83
84 # ----- MODBUS -----
85
86 class EscriitorModbus:
87     def __init__(self):
88         self.fila = modbus_queue
89         self.parar = threading.Event()
90         self.thread = None
91         self.cliente = conecta_plc()
92         self.rodando = True
93         self.ultima_verificacao = 0
94
95     def inicia(self):
96         self.parar.clear()
97         self.thread = threading.Thread(target=self._loop_escrita,
98                                         daemon=True)
99         self.thread.start()
100
101     def para(self):
102         self.parar.set()
103         if self.thread:
104             self.thread.join(timeout=2)
105         try:
106             self.cliente.close()
107         except:
108             pass
109
110     def _loop_escrita(self):
```

```
110         while not self.parar.is_set():
111             try:
112                 agora = time.time()
113                 if agora - self.ultima_verificacao > PLC_WRITE_INTERVAL:
114                     self._verifica_conexao()
115                     self.ultima_verificacao = agora
116
117             try:
118                 dados = self.fila.get(timeout=0.5)
119                 if dados is None:
120                     continue
121
122                 if self.cliente and self.cliente.is_socket_open():
123                     ok1 = escreve_bobina(self.cliente,
124                                         COIL_PEDESTRIAN, dados['pedestre'])
125                     ok2 = escreve_bobina(self.cliente,
126                                         COIL_HEARTBEAT, dados['heartbeat'])
127                     ok3 = escreve_bobina(self.cliente,
128                                         COIL_CAMERA_STATUS, dados['camera'])
129
130                     estado['plc_ok'] = ok1 and ok2 and ok3
131                     estado['camera_ok'] = dados['camera']
132
133                     print(f"PLC: PED={dados['pedestre']}
134                           HB={dados['heartbeat']}
135                           CAM={dados['camera']}")
136
137                 else:
138                     self._reconecta()
139
140             except queue.Empty:
141                 continue
142
143             except Exception as e:
144                 print(f"problema na thread do Modbus: {e}")
145                 time.sleep(0.1)
146
147     def _verifica_conexao(self):
148         if not self.cliente or not self.cliente.is_socket_open():
149             self._reconecta()
150
151     def _reconecta(self):
152         try:
153             if self.cliente:
154                 self.cliente.close()
155
156         except:
157             pass
158
159         self.cliente = conecta_plc()
```

```
152         if self.cliente and self.cliente.is_socket_open():
153             print("[PLC] Reconnectado com sucesso")
154         else:
155             print("[PLC] Falha na reconexao")
156
157 # ----- LEITOR DA STREAM -----
158
159 class LeitorStream:
160     def __init__(self, url):
161         self.url = url
162         self.frame_atual = None
163         self.lock = threading.Lock()
164         self.parar = threading.Event()
165         self.thread = None
166         self.conectado = False
167         self.fps_medido = deque(maxlen=30)
168         self.ultimo_frame_time = 0
169
170     def inicia(self):
171         self.parar.clear()
172         self.thread = threading.Thread(target=self._loop_leitura,
173                                         daemon=True)
174         self.thread.start()
175
176     def para(self):
177         self.parar.set()
178         if self.thread is not None:
179             self.thread.join(timeout=2)
180
181     def pega_frame(self):
182         with self.lock:
183             if self.frame_atual is None:
184                 return None
185             return self.frame_atual.copy()
186
187     def _loop_leitura(self):
188         buffer = b""
189
190         while not self.parar.is_set():
191             try:
192                 self.conectado = False
193                 with requests.get(self.url, stream=True,
194                                   timeout=HTTP_TIMEOUT) as r:
195                     r.raise_for_status()
196                     self.conectado = True
197
198                     for pedaco in r.iter_content(chunk_size=16384):
```

```
197         if self.parar.is_set():
198             break
199         if not pedaco:
200             continue
201
202         buffer += pedaco
203
204         while True:
205             inicio = buffer.find(JPEG_START)
206             fim = buffer.find(JPEG_END)
207
208             if inicio != -1 and fim != -1 and fim >
                inicio:
209                 jpg = buffer[inicio:fim + 2]
210                 buffer = buffer[fim + 2:]
211
212                 np_arr = np.frombuffer(jpg,
                    dtype=np.uint8)
213                 frame = cv2.imdecode(np_arr,
                    cv2.IMREAD_COLOR)
214
215                 if frame is not None:
216
217                     if APPLY_VERTICAL_FLIP:
218                         frame = cv2.flip(frame, 0)
219
220                     with self.lock:
221                         self.frame_atual = frame
222
223                     agora = time.time()
224                     if self.ultimo_frame_time > 0:
225                         self.fps_medido.append(1.0 /
                            (agora -
                                self.ultimo_frame_time))
226                         self.ultimo_frame_time = agora
227                     else:
228                         if len(buffer) > 1_000_000:
229                             buffer = buffer[-100_000:]
230                         break
231
232         except Exception as e:
233             self.conectado = False
234             print(f"perda conexao camera: {e}")
235             time.sleep(0.5)
236
237 # ----- PRE-PROCESSAMENTO -----
238
```

```

239 def prepara_frame(frame):
240     frame = cv2.resize(frame, (FRAME_W, FRAME_H),
241                          interpolation=cv2.INTER_LINEAR)
242
243     if APPLY_CONTRAST_BRIGHTNESS:
244         frame = cv2.convertScaleAbs(frame, alpha=ALPHA_CONTRAST,
245                                     beta=BETA_BRIGHTNESS)
246
247     return frame
248
249 # ----- METRICAS -----
250
251 def salva_metricas(tempo_captura, tempo_prep, tempo_yolo,
252                   tempo_modbus, tempo_total, fps, cpu,
253                   ram, pedestre, camera_ok, plc_ok):
254
255     try:
256         with open(metrics_file, 'a') as f:
257             f.write(f"{tempo_captura:.2f},{tempo_prep:.2f},"
258                    f"{tempo_yolo:.2f},{tempo_modbus:.2f},{tempo_total:.2f},"
259                    f"{fps:.2f},{cpu:.2f},{ram:.2f},"
260                    f"{int(pedestre)},{int(camera_ok)},{int(plc_ok)}\n")
261     except Exception as e:
262         print(f"Erro ao escrever metricas: {e}")
263
264 # ----- MAIN -----
265
266 def main():
267     try:
268         with open(metrics_file, 'w') as f:
269             f.write("TempoCaptura(ms),TempoPrep(ms),TempoYOLO(ms),"
270                    "TempoModbus(ms),TempoTotal(ms),FPS,CPU(%) ,RAM(%) ,"
271                    "Pedestre, CameraOK, PLCOK\n")
272             print(f"Arquivo de metricas criado: {metrics_file}")
273     except Exception as e:
274         print(f"Erro ao criar arquivo de metricas: {e}")
275
276     model = YOLO(MODEL_PATH)
277
278     stream = LeitorStream(STREAM_URL)
279     stream.inicia()
280
281     modbus = EscriitorModbus()
282     modbus.inicia()
283
284     ultima_escrita = 0.0
285     ultima_metrica = 0
286     contador_frames = 0

```

```

284 fps = 0.0
285 ultimo_fps_time = time.time()
286
287 #histerese
288 pedestre_estado = False
289 contador_on = 0
290 contador_off = 0
291 heartbeat = False
292
293 print("Rodando. Pressione 'q' para sair.")
294
295 try:
296     while True:
297         inicio_captura = time.time()
298         frame = stream.pegar_frame()
299         tempo_captura = (time.time() - inicio_captura) * 1000
300
301         if frame is None:
302             cv2.waitKey(1)
303             time.sleep(0.005)
304             continue
305
306         inicio_prep = time.time()
307         frame = prepara_frame(frame)
308         tempo_prep = (time.time() - inicio_prep) * 1000
309
310         inicio_yolo = time.time()
311         results = model.predict(
312             frame,
313             conf=CONF_THRES,
314             imgsz=YOLO_IMGSZ,
315             classes=[COCO_PERSON],
316             verbose=False,
317             device=YOLO_DEVICE
318         )
319         tempo_yolo = (time.time() - inicio_yolo) * 1000
320
321         pessoa_detectada = False
322         r0 = results[0]
323         if r0.bboxes is not None and len(r0.bboxes) > 0:
324             bboxes = r0.bboxes
325             for cls_id, conf, xxyy in zip(bboxes.cls.tolist(),
326                                         bboxes.conf.tolist(), bboxes.xxyy.tolist()):
327                 if int(cls_id) == COCO_PERSON:
328                     pessoa_detectada = True
329                     x1, y1, x2, y2 = map(int, xxyy)

```

```
330         cv2.rectangle(frame, (x1, y1), (x2, y2), (0,
331             255, 0), 2)
332         cv2.putText(frame, f"pessoa {conf:.2f}", (x1,
333             max(20, y1 - 8)),
334             cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
335             255, 0), 2)
336
337     #HISTERESE
338     if pessoa_detectada:
339         contador_on += 1
340         contador_off = 0
341     else:
342         contador_off += 1
343         contador_on = 0
344
345     if not pedestre_estado and contador_on >=
346         ON_FRAMES_REQUIRED:
347         pedestre_estado = True
348     elif pedestre_estado and contador_off >=
349         OFF_FRAMES_REQUIRED:
350         pedestre_estado = False
351
352     inicio_modbus = time.time()
353     agora = time.time()
354
355     if agora - ultima_escrita >= PLC_WRITE_PERIOD:
356         ultima_escrita = agora
357
358     heartbeat = not heartbeat
359
360     dados_modbus = {
361         'pedestre': pedestre_estado,
362         'heartbeat': heartbeat,
363         'camera': stream.conectado
364     }
365
366     try:
367         modbus_queue.put_nowait(dados_modbus)
368         estado['pedestre'] = pedestre_estado
369         estado['heartbeat'] = heartbeat
370         estado['pessoa_detectada'] = pessoa_detectada
371     except queue.Full:
372         pass
373
374     tempo_modbus = (time.time() - inicio_modbus) * 1000
375
376     #FPS
```

```
372         contador_frames += 1
373         if agora - ultimo_fps_time >= 1.0:
374             fps = contador_frames / (agora - ultimo_fps_time)
375             contador_frames = 0
376             ultimo_fps_time = agora
377
378         #METRICAS
379         ultima_metrica += 1
380         if ultima_metrica >= METRICS_WRITE_INTERVAL:
381             ultima_metrica = 0
382
383             cpu = psutil.cpu_percent()
384             ram = psutil.virtual_memory().percent
385             tempo_total = tempo_captura + tempo_prep + tempo_yolo +
386                 tempo_modbus
387
388             salva_metricas(
389                 tempo_captura, tempo_prep, tempo_yolo,
390                 tempo_modbus, tempo_total, fps,
391                 cpu, ram,
392                 pessoa_detectada,
393                 stream.conectado,
394                 estado['plc_ok']
395             )
396
397         #MOSTRA NA TELA
398         info1 = f"Pessoa: {pessoa_detectada}"
399         info2 = f"PLC: {'OK' if estado['plc_ok'] else 'OFF'}"
400         info3 = f"Camera: {'OK' if stream.conectado else 'OFF'}"
401         info4 = f"FPS: {fps:.1f}"
402
403         y = 25
404         for texto in [info1, info2, info3, info4]:
405             cv2.putText(frame, texto, (10, y),
406                         cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2)
407             y += 26
408
409         cv2.imshow("Modulo de processamento - Lucas Medeiros",
410                   frame)
411
412         if cv2.waitKey(1) & 0xFF == ord('q'):
413             break
414
415     finally:
416         stream.para()
417         modbus.para()
418         cv2.destroyAllWindows()
```

```
416         print(f"metricas salvas em: {metrics_file}")
417
418 if __name__ == "__main__":
419     main()
```

Excerto de código B.1 – Código de configuração do bloco de processamento

APÊNDICE C – LÓGICA LADDER

A seguir, apresenta-se a descrição da lógica *Ladder* implementada no controlador semafórico. A lógica implementa uma sequência temporizada de estados, na qual a presença de pedestres faz com que o controlador execute um ciclo completo de travessia, envolvendo as fases de verde, amarelo e vermelho para os veículos, bem como a alternância correspondente dos sinais de pedestres.

A implementação foi estruturada de forma modular, separando o controle dos estados do semáforo, a temporização entre as fases e os mecanismos de supervisão da comunicação com o sistema de processamento, o que permite que cada etapa do funcionamento do controlador seja executada de forma sequencial e determinística, característica essencial em aplicações de automação industrial.

O funcionamento do controlador inicia a partir da variável *TEM_PED*, recebida por meio do protocolo *Modbus/TCP* a partir do sistema de visão computacional. Essa variável representa a presença ou ausência de pedestres na região monitorada pela câmera. Sempre que sua condição é alterada, a lógica *Ladder* passa a decidir qual sequência de estados deverá ser executada pelo semáforo, alternando entre o ciclo convencional e o ciclo adaptativo previsto para o projeto.

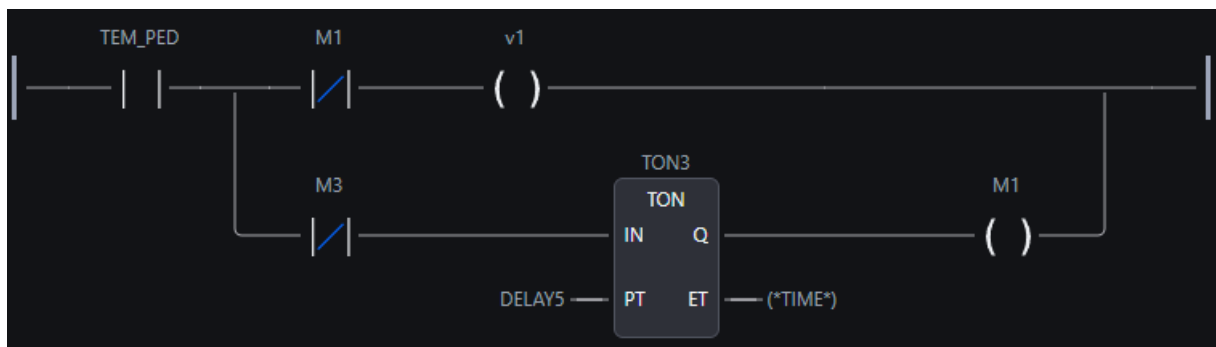


Figura 33 – Temporizador do semáforo veicular (verde). Fonte: Autoria própria.

A temporização das fases foi implementada utilizando blocos funcionais do tipo *TON* (*Timer On Delay*). Foram empregados três temporizadores principais responsáveis por controlar a duração dos estados verde, amarelo e vermelho do semáforo veicular, conforme Figuras 33, 34 e 35. Os tempos de cada fase são definidos pelas variáveis *DELAY5* e *DELAY2*, correspondendo, respectivamente, aos intervalos de cinco segundos para as fases principais e dois segundos para a fase de transição amarela. Dessa forma, a sequência temporal do semáforo é executada automaticamente, sem depender de temporizações implementadas externamente ao controlador.

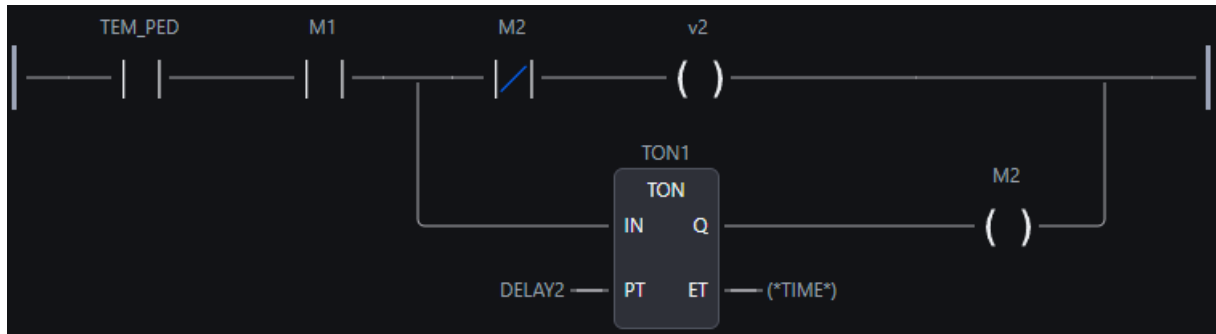


Figura 34 – Temporizador do semáforo veicular (amarelo). Fonte: Autoria própria.

Para organizar a evolução entre as diferentes etapas do ciclo semafórico foram utilizadas variáveis internas de memória, denominadas $M1$, $M2$ e $M3$, além das variáveis auxiliares $v1$, $v2$, $v3$, $v1p$ e $v3p$ (Figura 36). Essas variáveis funcionam como estados intermediários da máquina de estados implementada em *Ladder*, permitindo que cada temporizador seja ativado apenas quando as condições da etapa anterior forem satisfeitas. Essa abordagem simplifica a lógica de programação e evita que múltiplos estados do semáforo sejam acionados simultaneamente.

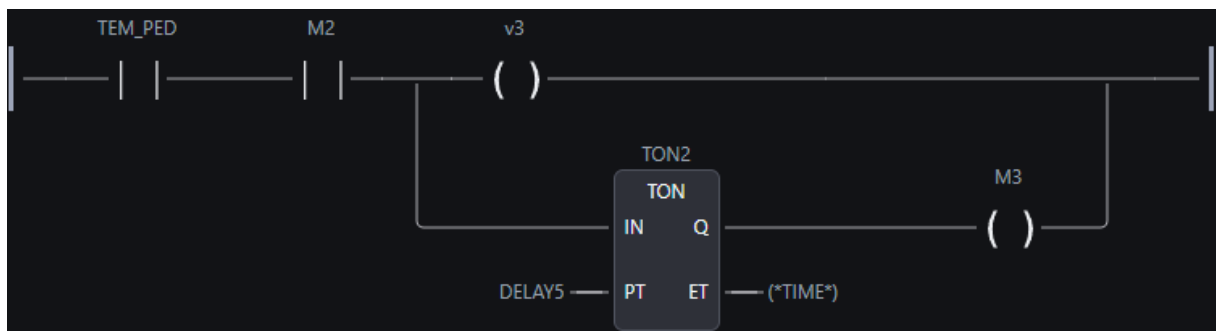


Figura 35 – Temporizador do semáforo veicular (vermelho). Fonte: Autoria própria.

A partir dessas variáveis intermediárias são comandadas diretamente as saídas digitais do *ESP32* responsáveis pelos *LEDs* que simulam as lâmpadas do semáforo. As variáveis *VERDE*, *AMARELO* e *VERMELHO* (Figuras 37, 38 e 39, respectivamente) controlam o semáforo veicular, enquanto *VERDE_PED* e *VERMELHO_PED* (Figura 39) comandam o semáforo dos pedestres. A lógica implementada garante que apenas as combinações válidas de estados possam ocorrer, impedindo conflitos entre as sinalizações de veículos e pedestres.

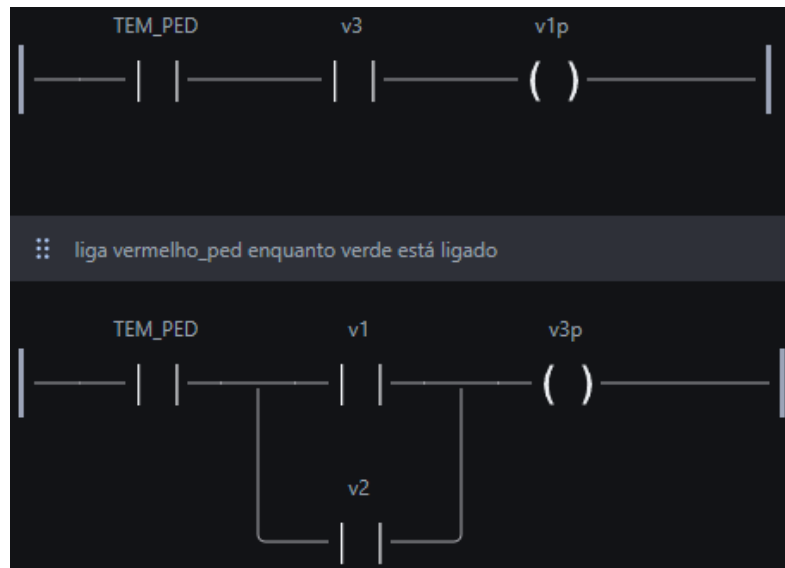


Figura 36 – Lógica do semáforo de pedestres. Fonte: Autoria própria.

Além da lógica de operação normal, foi implementado um mecanismo de supervisão da comunicação entre o controlador e o sistema de processamento. Para essa finalidade, o programa em *Python* envia continuamente duas variáveis adicionais ao controlador: o sinal de *Heartbeat*, responsável por indicar que o sistema de processamento permanece em execução, e a variável *CameraOk*, que informa se a transmissão de imagens da *ESP32-CAM* permanece ativa.

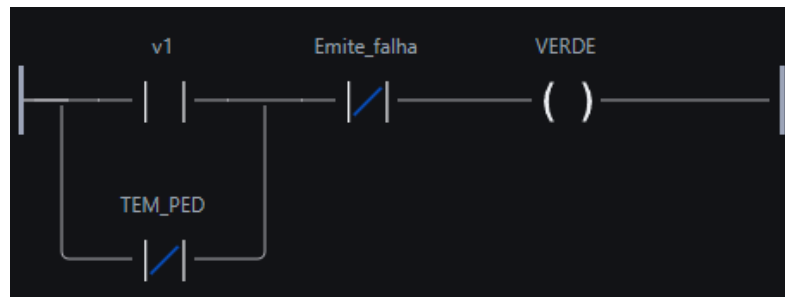


Figura 37 – Lógica final para acionamento dos LEDs do semáforo veicular (verde). Fonte: Autoria própria.

O monitoramento do sinal de *Heartbeat* foi implementado utilizando os blocos detectores de borda de subida (*R_TRIG*) e de borda de descida (*F_TRIG*), conforme Figura 40. Como o programa de processamento alterna continuamente o valor dessa variável entre os estados lógico 0 e 1, cada transição gera um pulso de curta duração, indicando ao controlador que a comunicação continua ativa. Esses pulsos são reunidos na variável *HB_mudou*, utilizada como indicação de atualização do sistema de processamento.

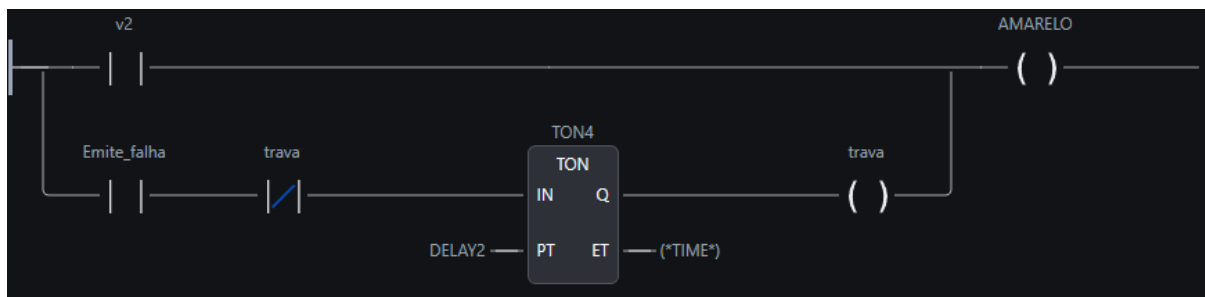


Figura 38 – Lógica final para acionamento dos LEDs do semáforo veicular (amarelo). Fonte: Autoria própria.

A variável *HB_mudou*, juntamente com o estado da variável *CameraOk*, alimenta um temporizador adicional denominado *TON_heartbeat*. Sempre que o programa de processamento deixa de alternar o heartbeat durante um intervalo superior ao tempo configurado ou quando a câmera deixa de transmitir imagens, o temporizador completa sua contagem e aciona a variável *Emite_falha*. Essa variável indica uma falha de comunicação entre os módulos do sistema.

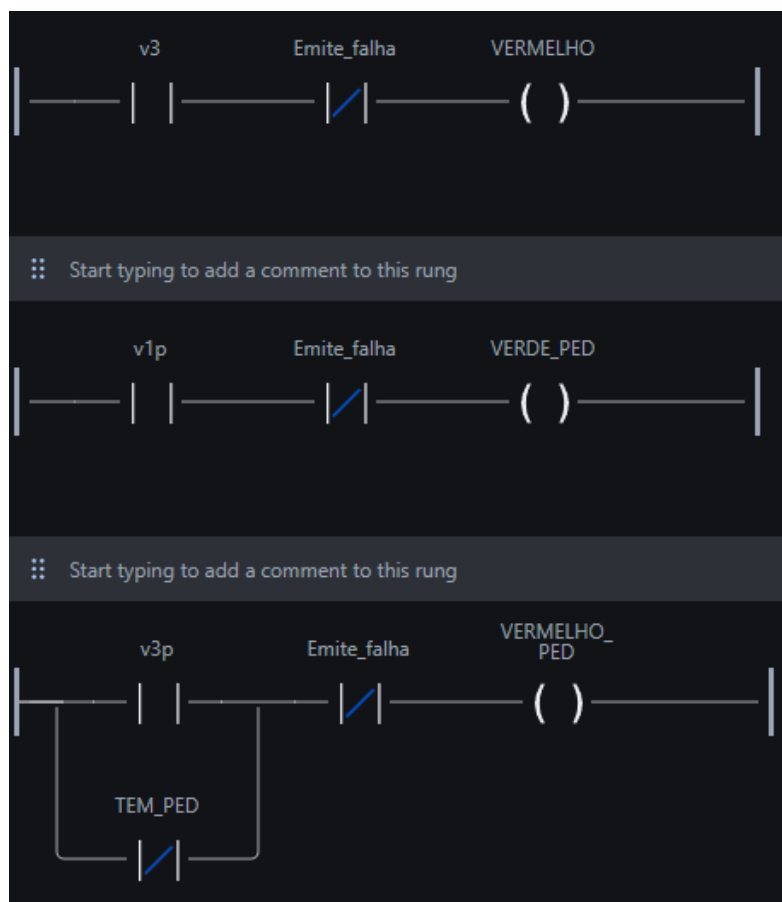


Figura 39 – Lógica final para acionamento dos LEDs do semáforo veicular (vermelho) e de pedestres. Fonte: Autoria própria.

Uma vez detectada essa condição, a variável *Emite_falha* passa a atuar diretamente sobre a lógica principal do controlador. Durante sua ativação, os contatos normalmente fechados associados a essa variável interrompem a sequência normal de estados do semáforo, impedindo novas transições do ciclo convencional. Em paralelo, um temporizador adicional (*TON4*) e a variável interna *trava* produzem o efeito de intermitência necessário para acionar o *LED* amarelo em modo piscante (Figura 38), caracterizando o comportamento de estado seguro do controlador.



Figura 40 – Lógica para acionamento do sistema de segurança. Fonte: Autoria própria.